目次

Module 1. Python 技術及開發環境介紹 1
認識 Python 應用 1
建置 Python 開發環境 1
安裝 Python 單一環境（上課不會用到）..... 2
安裝 Anaconda 開發軟體（上課建議使用）..... 6
如何執行 Python 程式 14
使用 IDE 14
建立開發專案的資料夾 20
讓專案資料夾使用 Conda 環境..... 22
建立與執行 .py 檔案 24
參考資料 25
Module 2.資料型態與變數 26
資料型態 26
變數..... 27
變數命名規則、關鍵字與縮排..... 28
變數命名規則..... 28
關鍵字 28
縮排 29
運算子..... 29
算術運算子..... 30
賦值運算子..... 30
比較運算子..... 31
邏輯運算子..... 32
成員運算子..... 32
身分運算子..... 33
位元運算子..... 33
運算子優先順序 34
註解 34
參考資料 36
Module 3.控制結構..... 37
循序..... 37
選擇..... 37
重複..... 38
Module 4.字串、串列、元組、字典、集合 42
字串..... 42
補充：字串格式化..... 43
補充：input() 與 eval() 46

串列 47
補充：序列物件分割（Slicing） 49
補充：List Comprehension 51
補充：enumerate() 51
元組 52
字典 53
補充：Dictionary Comprehension 55
補充：dict.items() 56
集合 56
參考資料 58
Module 5.函式 60
基本用法 60
變數傳遞 60
區域變數與全域變數 61
匿名函式 Lambda 63
參考資料 65
Module 6.物件導向簡介 66
基本認識物件導向 66
類別 66
方法 66
屬性 67
繼承 68
參考資料 70
Module 7.例外處理 71
認識例外處理 71
try-except 陳述 71
例外處理 raise 陳述 73
補充：logging 73
參考資料 76
Module 8. 檔案處理 78
open() 函式 78
讀取檔案 78
寫入檔案 79
刪除檔案 79
參考資料 79
Module 9. 模組 81
定義模組 81
使用模組 81

補充：__name__ 內建變數的用法 82
補充：模組不在當前路徑下的處理方式 84
參考資料 85
Module 10. 指令列參數 86
sys.argv 86
argparse.ArgumentParser 87
參考資料 88
Module 11. 修飾子 Decorator 89
基本概念 89
使用語法糖的修飾子 90
修飾子可以疊加..... 90
參考資料 91
Module 12. 安裝套件 92
pip 指令 92
參考資料 94

- GitHub 專案連結
  https://github.com/telunyang/python_basics

Module 1. Python 技術及開發環境介紹

認識 Python 應用

Python 是一種跨平台 (Cross-platform)、開放原始碼 (Open Source) 、擁有大量社群 (Communities)、物件導向 (Object-Oriented Language) 的程式語言，由 Guido van Rossum (Python 最初設計者兼程式架構師) 在 1989 年底發明。Python 名稱的發想，是來自 Rossum 喜愛的電視喜劇《Monty Python's Flying Circus》。

Python 的語法簡潔、明確，可以整合許多工具來開發各式各樣的功能與服務，例如網站建置、資料分析與視覺化、機器學習、深度學習、視窗表單設計、自動化、遊戲開發、資料庫操作等，都是 Python 的強項。

Python 目前主流是 3.x 版本，過去的 2.x 版本已經不再被官方維護，強烈建議大家安裝 3.x 版來進行程式開發與學習。

建置 Python 開發環境

Python 的環境建置來源，通常分為官方網站 (https://www.python.org/) 與專門用於科學計算的 Anaconda (https://www.anaconda.com/) 或 MiniConda (https://docs.conda.io/en/latest/miniconda.html) ，個人會建議使用 Anaconda，原因如下：

1. 可透過圖形化介面操作，開發用的套件 (Packages) 清單、環境版本控制等，管理上較為容易。
2. 常用工具與科技計算用的會幫你事先安裝完成，對初學者較為友善。當同學慢慢熟悉開發流程的時候，也可以使用終端機 (Terminal) 來安裝適合自己的工具或套件。

Anaconda 也會有它的缺點，例如：

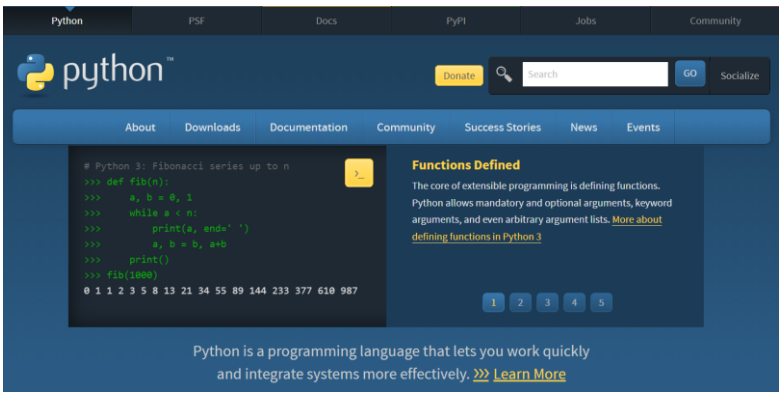
1. 下載安裝的檔案很大、時間很久，對進階的開發者而言，預先下載的工具不見得是開發者要的，換句話說，一開始會有很多短時間用不著，或是要學習到一個階段的時候，才有機會用到的工具。對於初學者來說，透過 Anaconda 建立第一個開發環境是很重要的，繁雜的流程，容易讓人望而卻步。
2. 安裝工具 conda & pip 容易讓人搞不清楚。conda 與 pip 都可以安裝開發環境當中的套件：
   - conda 主要是安裝開發的「環境」(Environment)，例如透過

conda 選定一個 python 版本，然後給予自訂名稱作為開發環境的識別，以後可以任意切換不同 python 版本的環境。

- pip 純粹是套件安裝與管理工具，在不同的 conda 環境中，針對環境使用的 python 版本，來決定下載、安裝哪個 python 版本適用的套件。

安裝 Python 單一環境（上課不會用到）

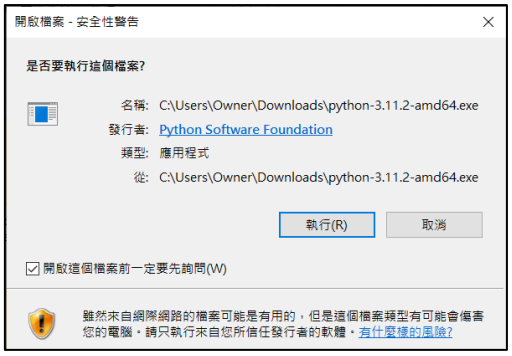
官方網站（https://www.python.org/）



圖：python.org 首頁

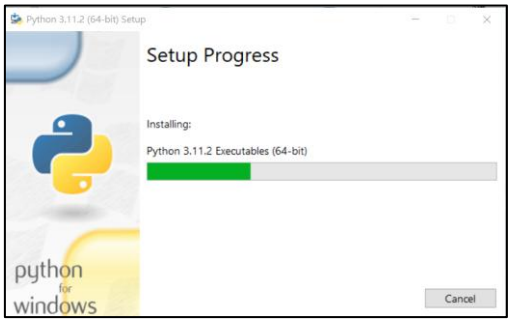
圖：游標移至 Download，看到 Python 下載按鈕，直接點它下載安裝程式

圖：快速點兩下，執行安裝程式

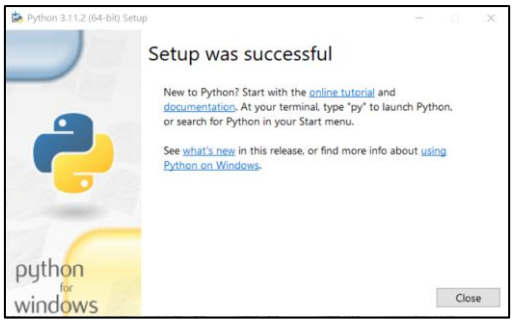


圖：若有安全性警告，直接按下「執行」

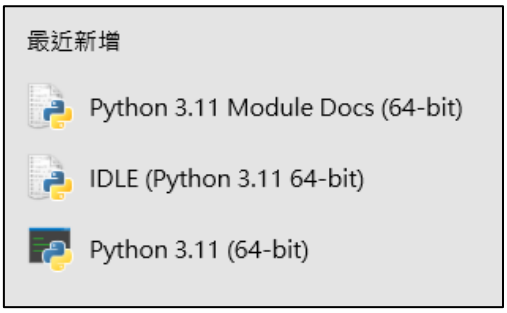
圖：打勾「Add python.exe to PATH」，按下上方的 Install Now
(若有安全性提示，可以按下同意)



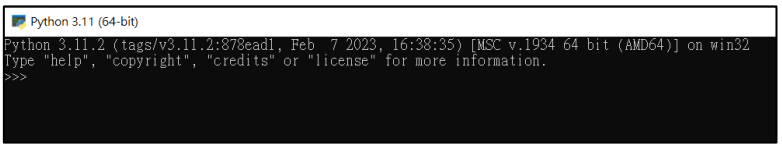
圖：等待安裝過程



圖：安裝成功畫面



圖：到 Windows 選單，尋找最近新增，選擇「Python 3.xx (64-bit)」

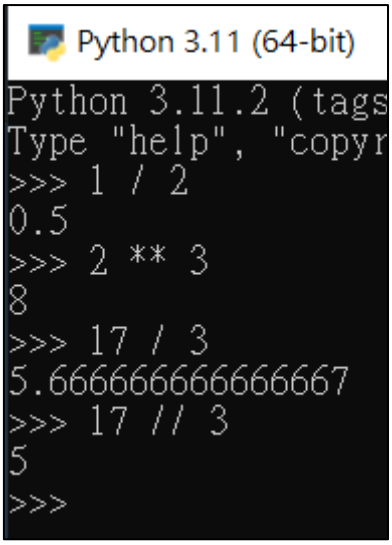


圖：python 指令操作介面

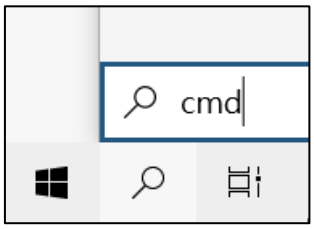
範例

>>> 1 / 2 (輸入完，按下 Enter)
0.5
>>> 2 ** 3 (輸入完，按下 Enter)
8
>>> 17 / 3 (輸入完，按下 Enter)
5.666666666666667
>>> 17 // 3 (輸入完，按下 Enter)
5

表：小試一下

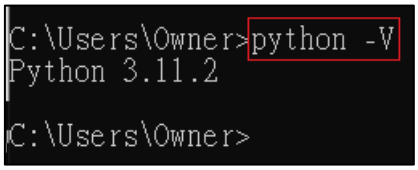


圖：執行結果。可輸入 quit() 或 exit()，再按 Enter 離開



圖：搜尋 cmd，按下 Enter，進入命令提示字元

圖：在命令提示字元輸入「python」後，按下 Enter，可使用 python



圖：可以在命令提示字元輸入「python -V」，按下 Enter，
得知當前 Python 的版本
若是安裝過程中，沒有打勾「python.exe to PATH」，
無法在命令提示字元中使用 python 指令

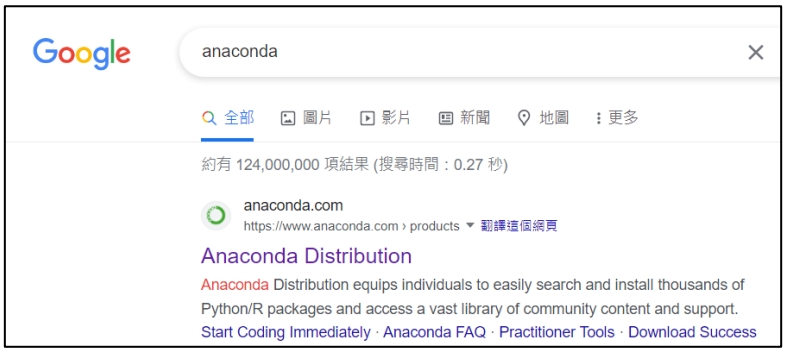
安裝 Anaconda 開發軟體（上課建議使用）

安裝 Anaconda

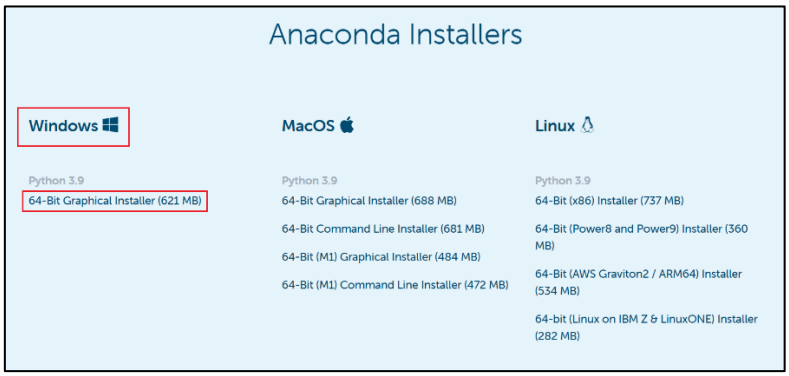
參考連結

[1] Anaconda - Individual Edition

https://www.anaconda.com/products/individual



圖：可以先至 google 檢索「anaconda」，找到 Anaconda Distribution 的連結

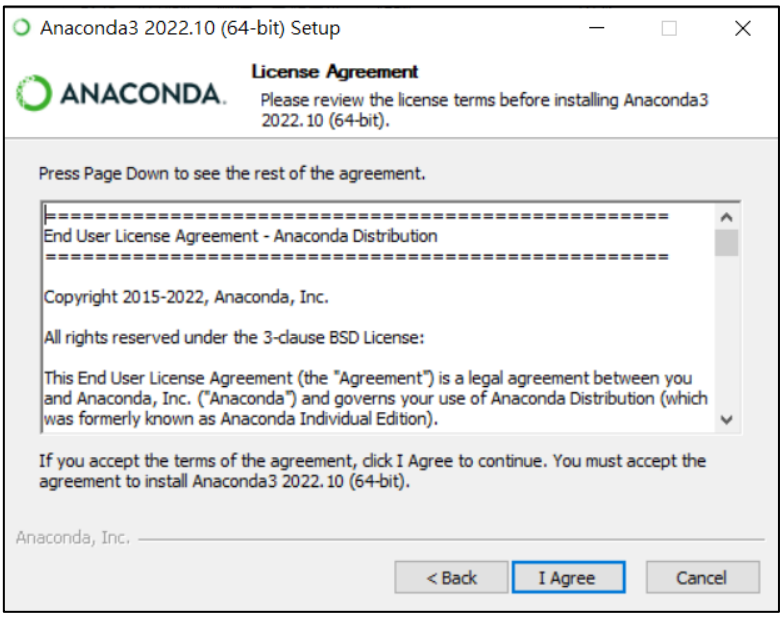


圖：在 Windows 選項下，選擇 64-Bit Graphical Installer，並下載下來

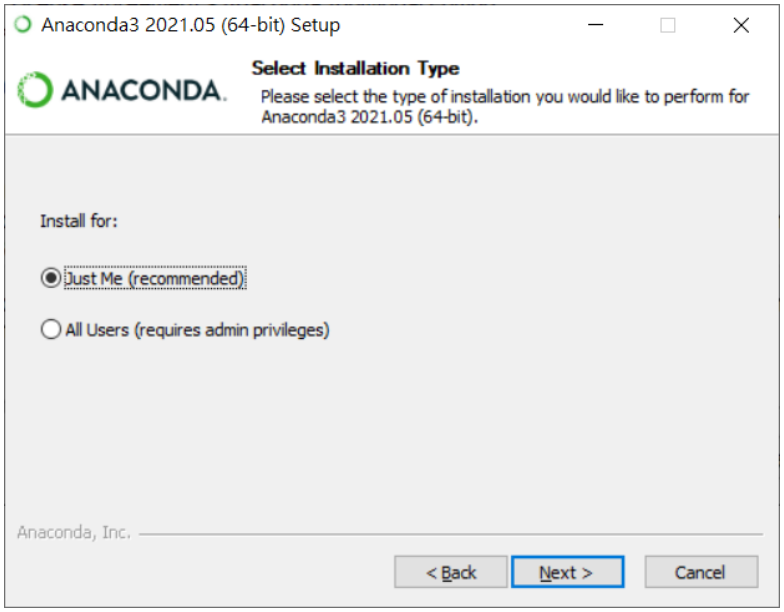
Anaconda3-202
2.10-Windows-x
86_64.exe

圖： 下載後，快速點兩下進行安裝
有安全性警告，可按下執行或同意

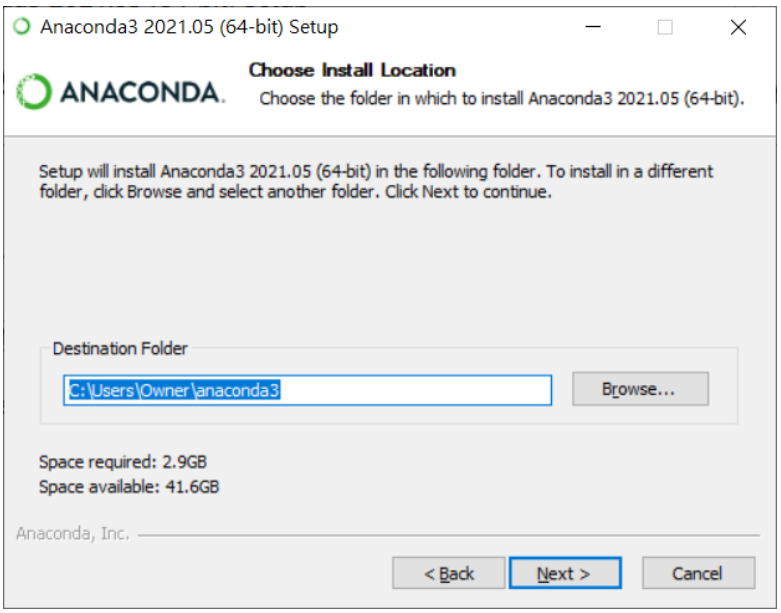
圖： 按下「Next」



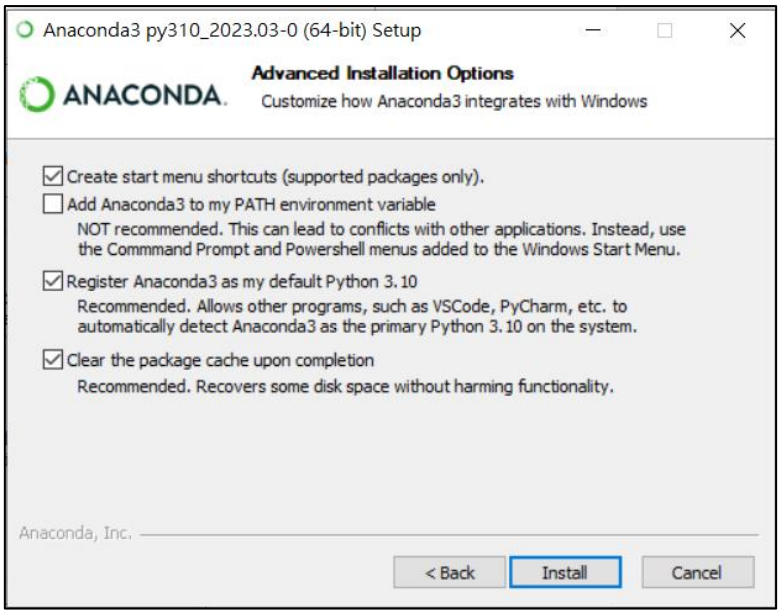
圖：按下「I Agree」



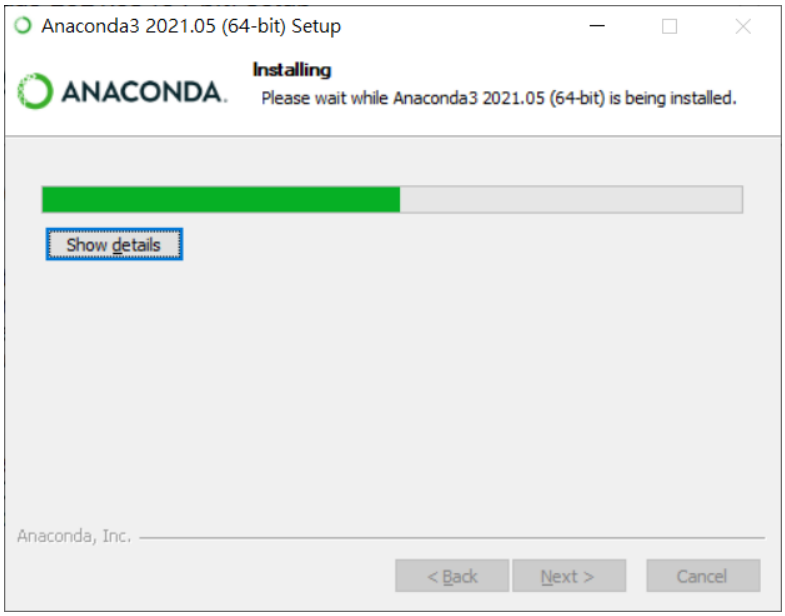
圖：依需求選擇後，按下「Next」



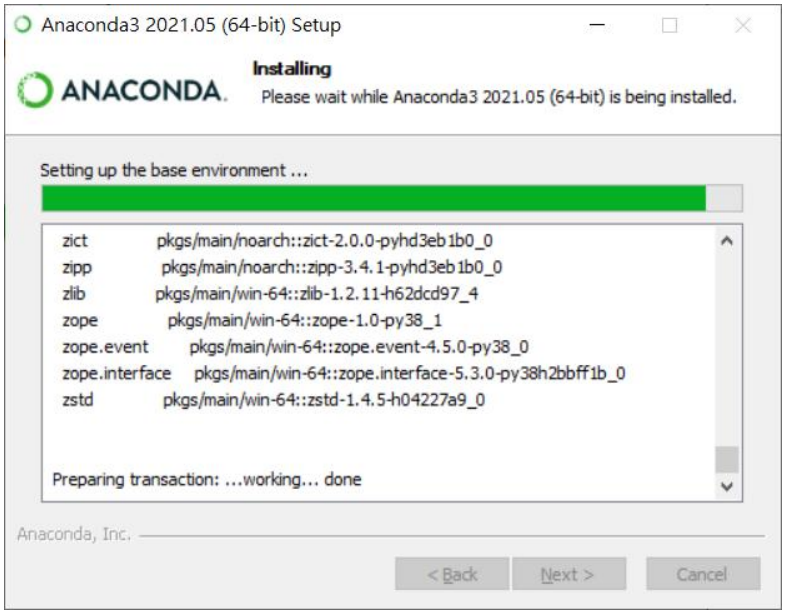
圖：安裝位置會在**使用者資料夾中的 anaconda3**，依需求設定，按下「Next」



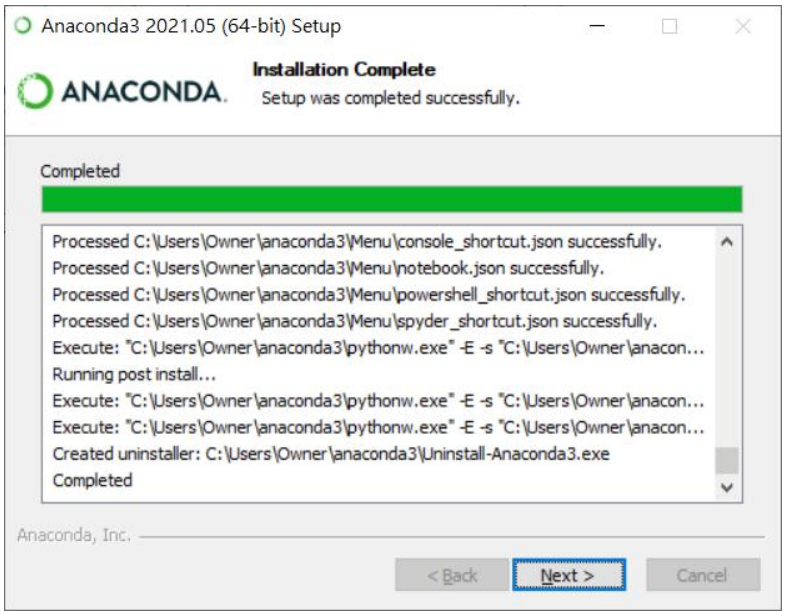
圖：依需求選擇（最後一個建議打勾，省空間），按下「Install」，進行安裝



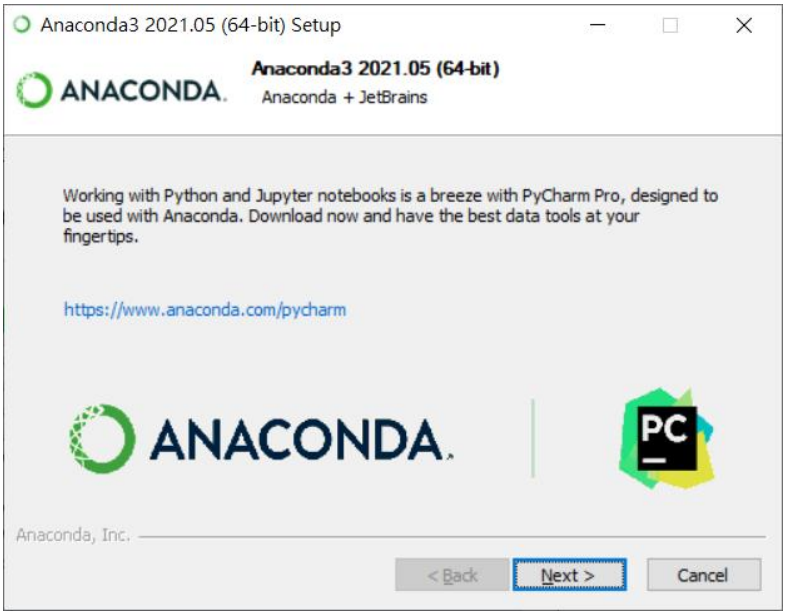
圖：安裝過程，需要一段時間



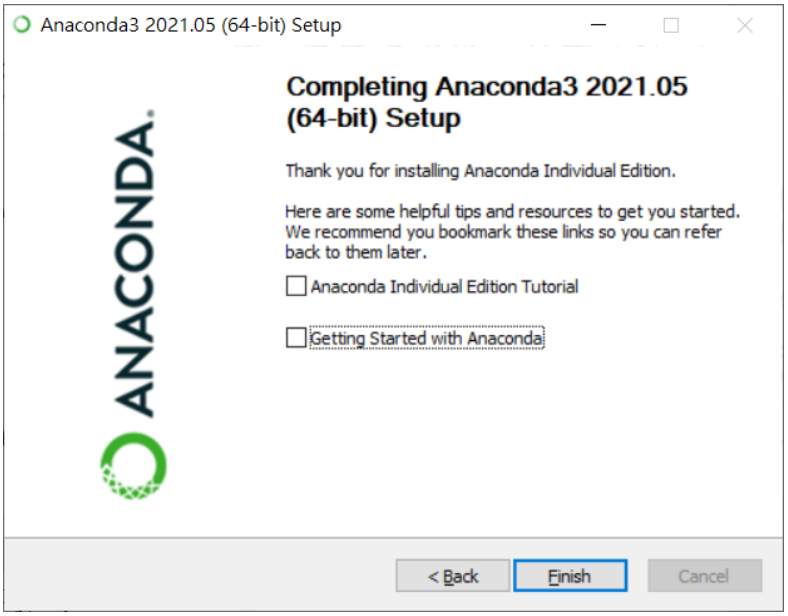
圖：按下「Show details」，會看到安裝過程



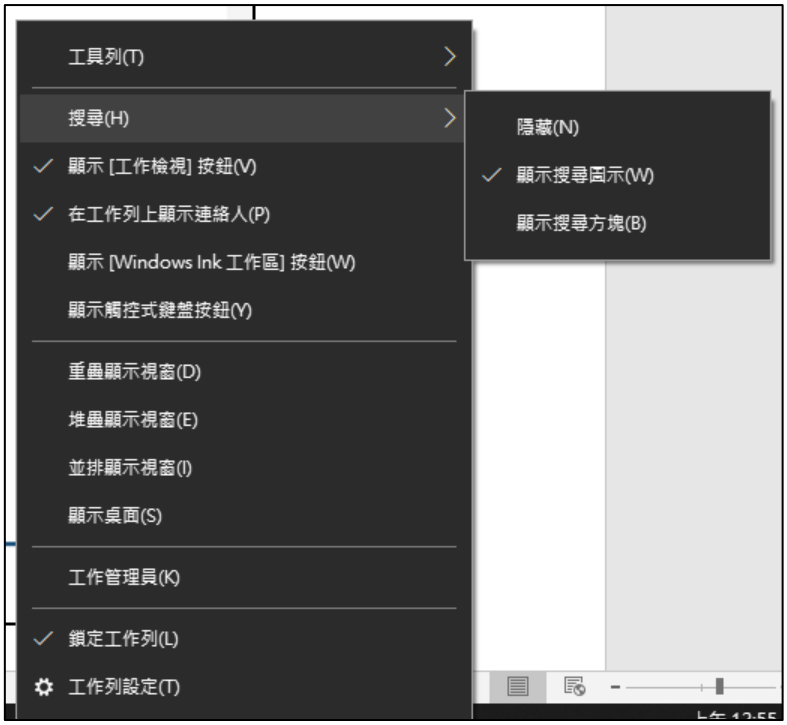
圖：安裝完成後，按下「Next」



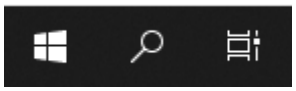
圖：按下「Next」



圖：取消勾選圖片中的兩個選項後，按下「Finish」



圖：顯示搜尋圖示



圖：搜尋圖示類似放大鏡，按下搜尋圖示

圖：搜尋「anaconda prompt」，按下「Anaconda Prompt (anaconda3)」

圖：出現 Anaconda Prompt，類似 Windows 的命令提示字元

圖：輸入「python」，進入 python 執行環境

圖：輸入「print("Hello, world")」，輸出「Hello, world」，python 安裝成功

圖：按下「exit()」回到指令輸入的環境

安裝、切換與刪除 Conda 環境 (Environment)

預設是 (base)，如果有切換環境的需求，例如手上處理著不同 Python 版本的開發專案，需要不時切換版本來開發，此時可以建立一到多個 Conda 環境，需要的時候可以切換，不需要的時候可以刪除。

在執行以下的指令前，需要確認目前是否在 Anaconda Prompt 當中，或是支援 Conda 的 Terminal 環境。終端機顯示預設路徑時，最前面會有 (base)，代表目前正在預設的 Conda 環境當中。

安裝 Conda 環境
範例 `conda create --name 自訂環境名稱 python=版本號`

```
conda create --name basic python=3.12
```

列出所有 Conda 環境
conda env list

離開當前的 Conda 環境
conda deactivate

切換 / 啟動 Conda 環境
conda activate basic

刪除 Conda 環境
conda remove -n basic --all

如何執行 Python 程式

使用 IDE

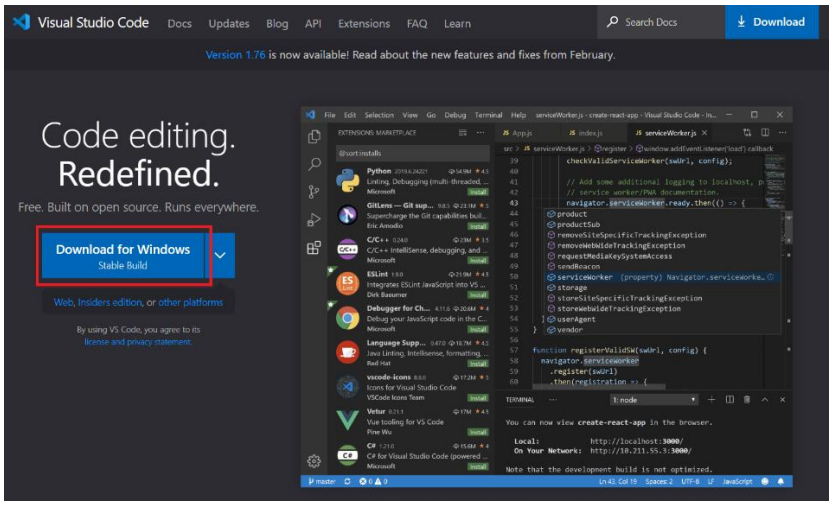
整合式開發環境 (Integrated Development Environment，IDE) 可以協助開發者方便使用類似記事本的程式碼輸入介面，側欄又有類似檔案總管的管理功能。常見的 Python IDE 有：

1. Visual Studio Code
2. Eclipse with PyDev
3. Sublime Text
4. Atom
5. Spyder
6. PyCharm
7. Vim
8. Emacs
9. Thonny
10. Wing
11. Google Antigravity
12. Cursor

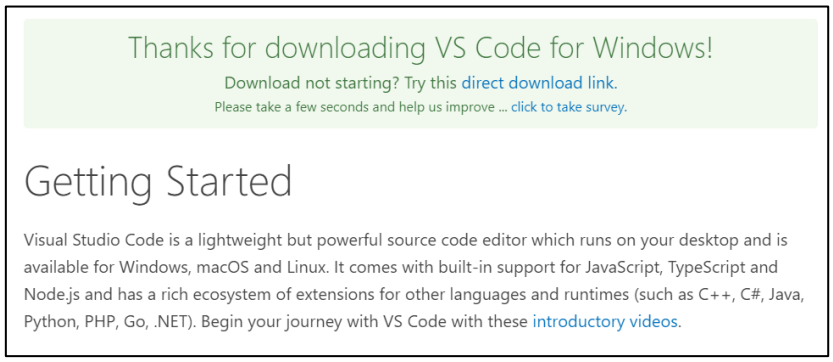
在課程中，我們選用「Visual Studio Code」(簡稱為 VS code) 來作為課程使用的 IDE。VS code 是微軟開發的跨平台 (Cross-platform) 程式

開發工具，除了開發程式之外，還可以使用豐富的擴充功能 (Extensions) 來提升開發的效率。

圖：在 google 搜尋「vs code」，進入 vs code 首頁

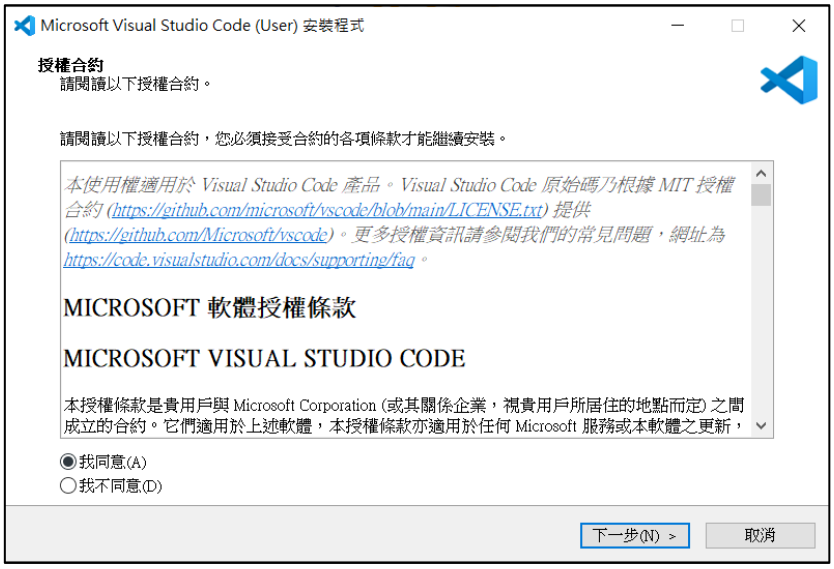


圖：網站會偵測使用者的作業系統，依需求選擇後，按下 Download for XXX

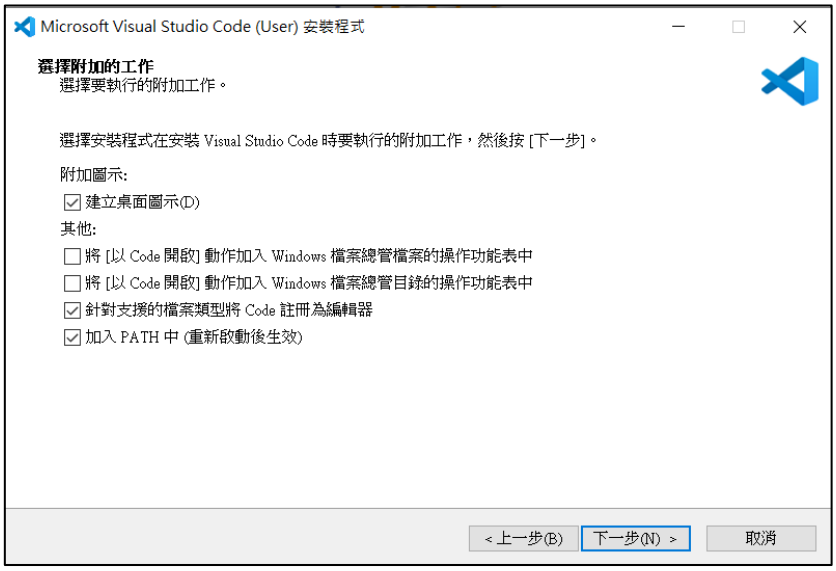


圖：看到這個畫面，安裝檔案會自動下載，
沒有下載的話，可以按下「direct download link」

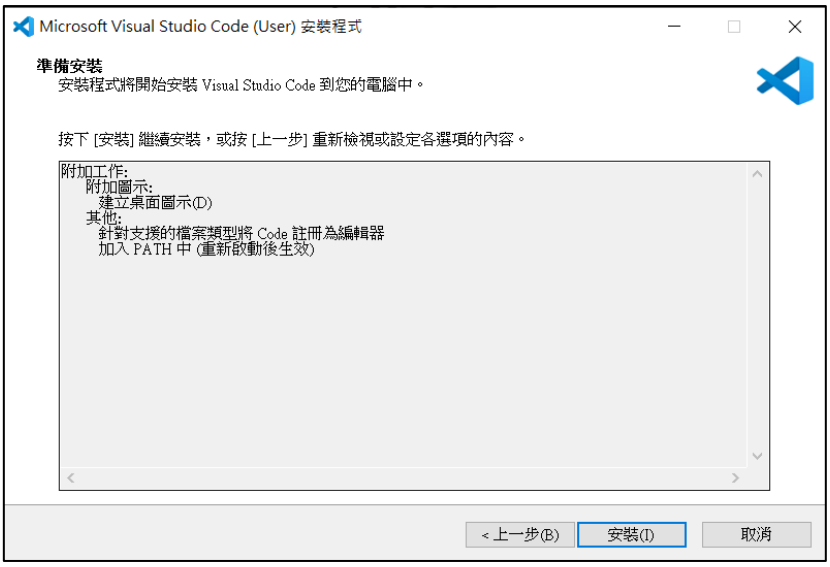
圖：快速連點兩下安裝，有安全性警告就按執行或同意



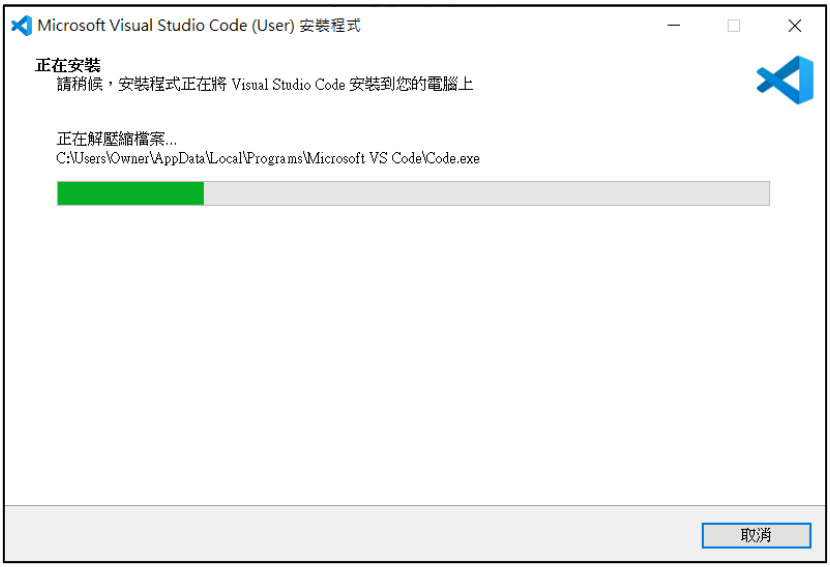
圖：勾選「我同意」，按「下一步」



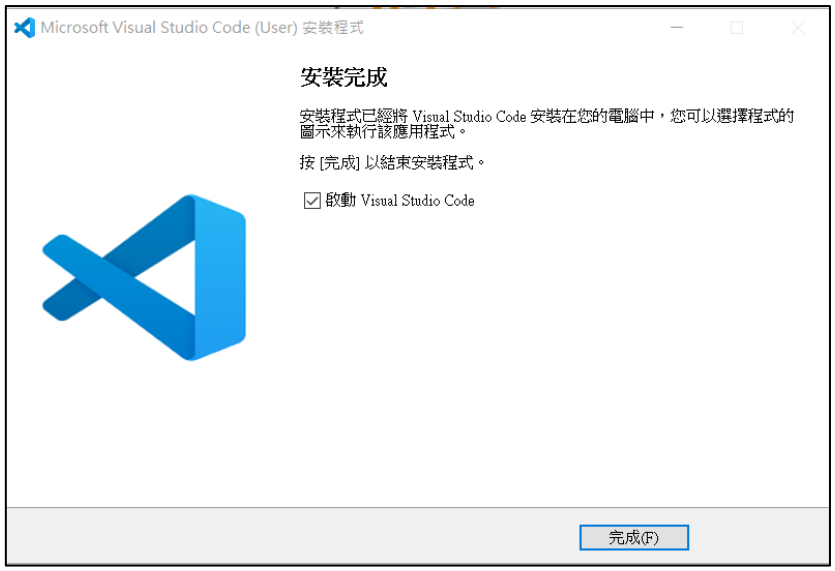
圖：勾選「建立桌面圖示」，按「下一步」


Microsoft Visual Studio Code (User) 安裝程式
準備安裝
安裝程式將開始安裝 Visual Studio Code 到您的電腦中。
按下 [安裝] 繼續安裝，或按 [上一步] 重新檢視或設定各選項的內容。
附加工作:
附加圖示:
建立桌面圖示(D)
其他:
針對支援的檔案類型將 Code 註冊為編輯器
加入 PATH 中 (重新啟動後生效)
< 上一步(B)
安裝(I)
取消

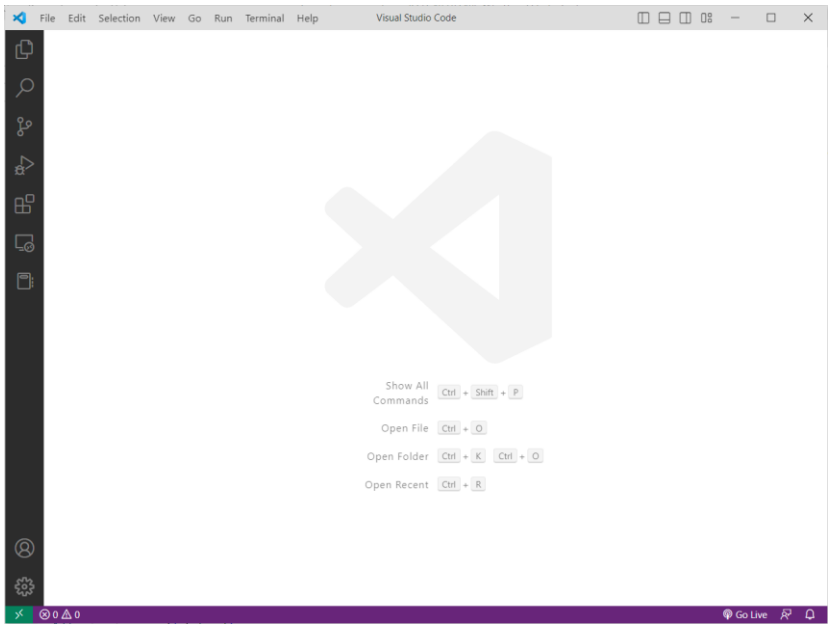
圖：按下「安裝」


Microsoft Visual Studio Code (User) 安裝程式
正在安裝
請稍候，安裝程式正在將 Visual Studio Code 安裝到您的電腦上
正在解壓縮檔案...
C:\Users\Owner\AppData\Local\Programs\Microsoft VS Code\Code.exe
取消

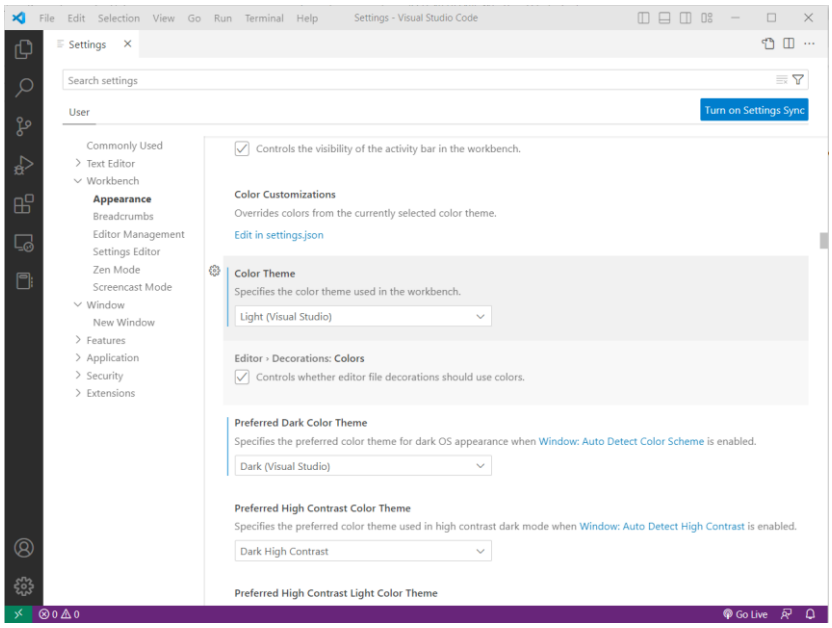
圖：安裝過程



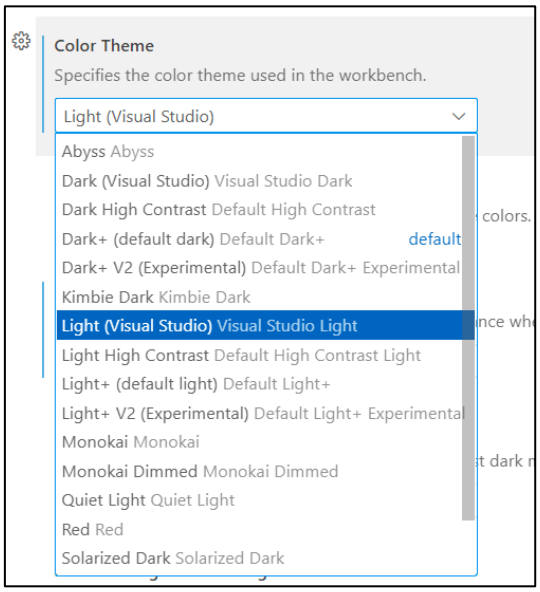
圖：勾選「啟動 Visual Studio Code」，按下「完成」



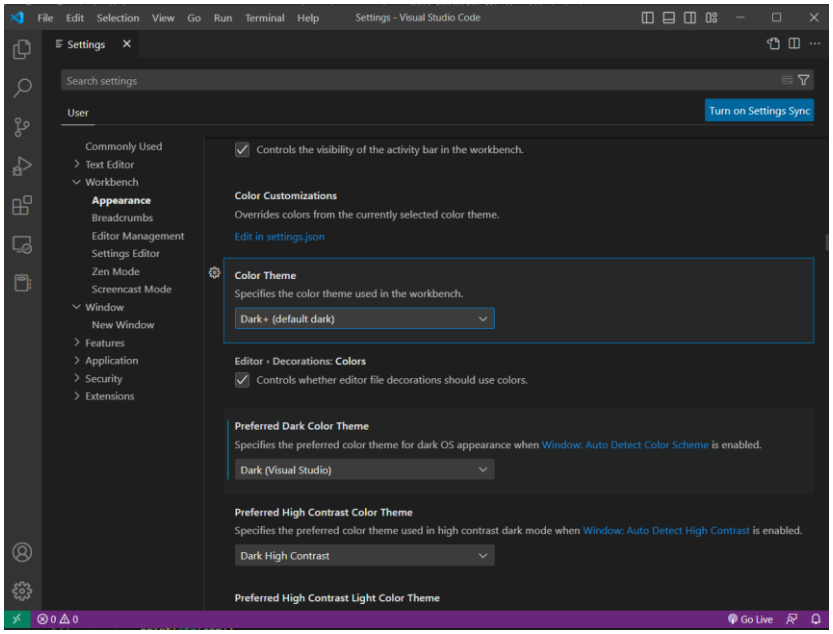
圖：VS code 的畫面



圖：如果想要換佈景顏色，按下「Ctrl + ,」來開啟設定

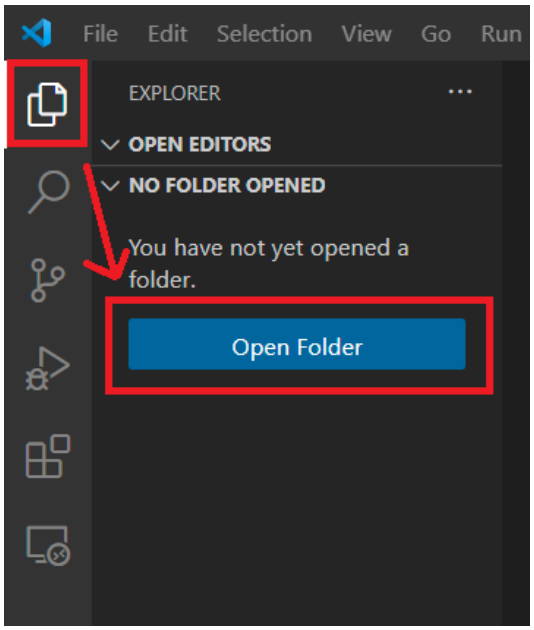


圖：在 Color Theme 下方的選單中，選擇偏好的佈景，例如選擇「Dark+」

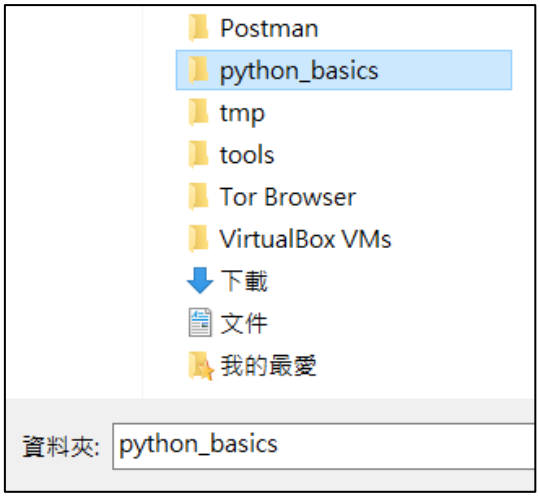


圖：選擇「Dark+」，可以立即套用選擇的佈景，
可以把左上方的頁籤 Settings 關掉（透過按下 Settings 右邊的 x 圖示）

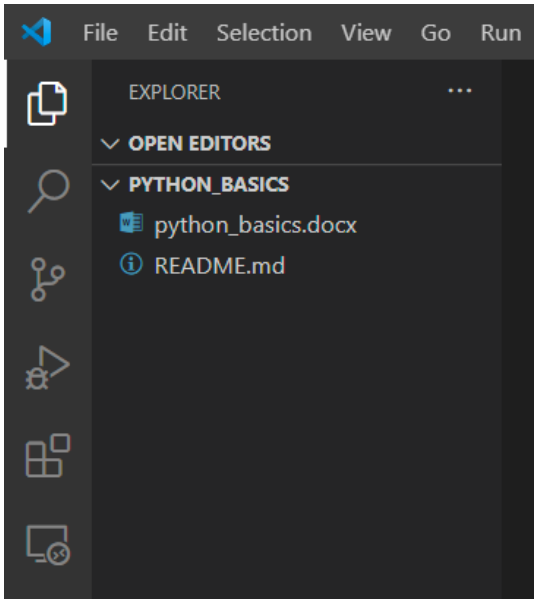
建立開發專案的資料夾



圖：按下左上角檔案總管（Explorer），再按下開啟資料夾（Open Folder），

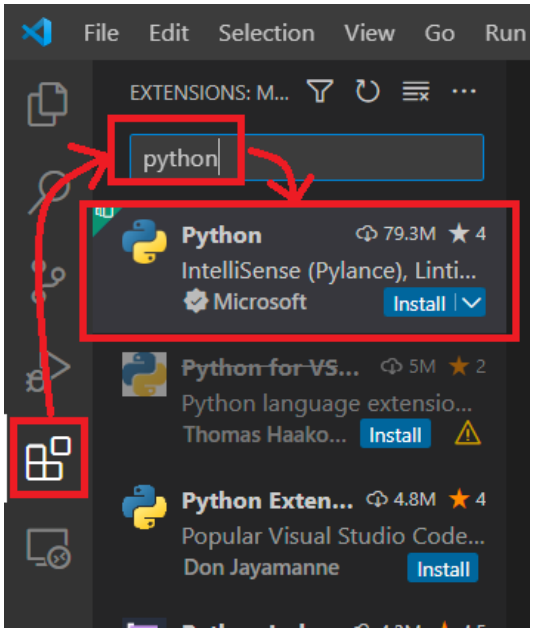


圖：選擇一個已存在的資料夾，或是手動新增，
再選擇新資料夾作為專案資料夾，按下「選擇資料夾」



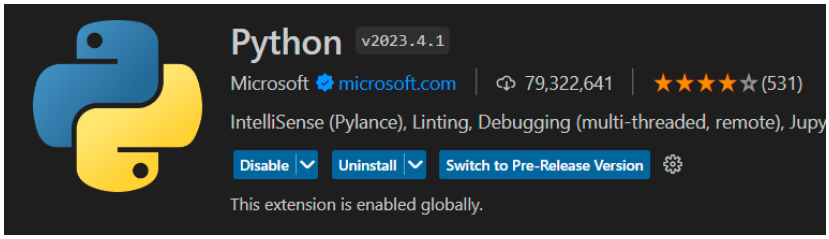
圖：選擇資料夾後的呈現方式

讓專案資料夾使用 Conda 環境

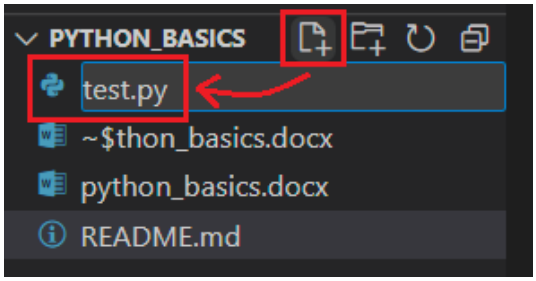


圖：按下擴充功能（Extensions），在上方輸入「python」，點選「Python」

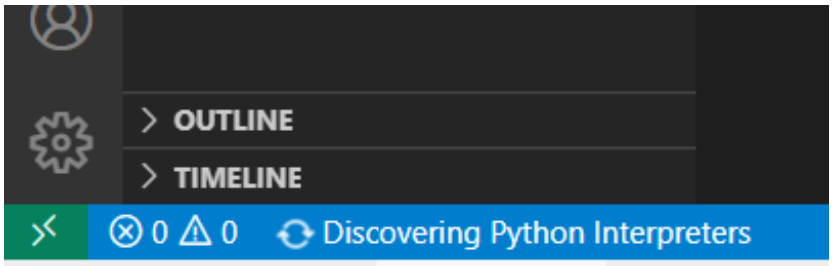
圖：按下 Install



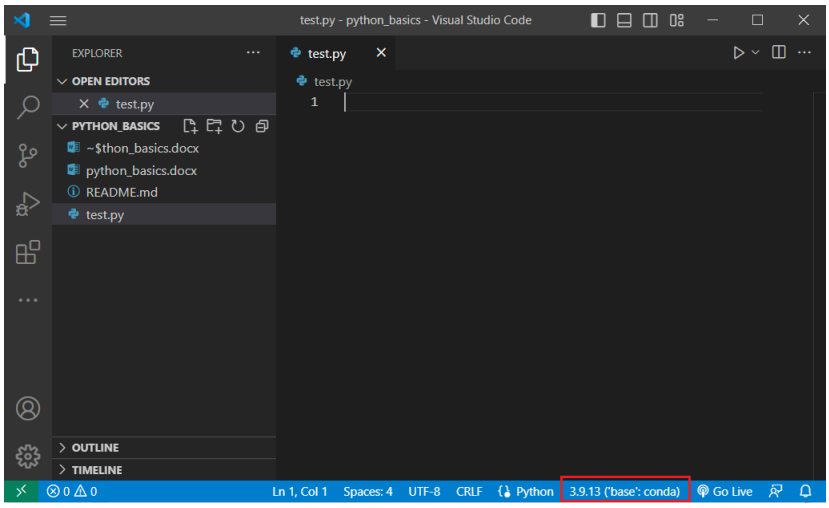
圖：出現 Uninstall 字眼，代表安裝完成，關閉頁籤，回到專案資料夾



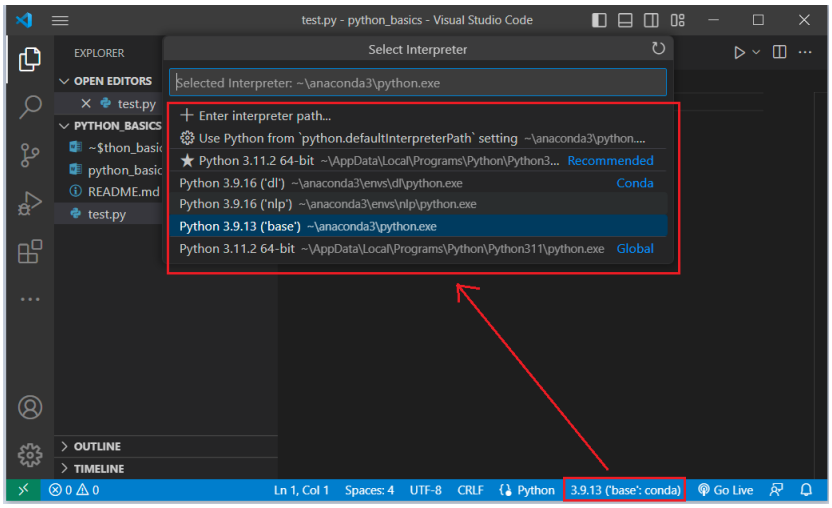
圖：新增檔案，輸入檔案為 test.py，按下 Enter



圖：開啟 .py 的檔案後，VS code 自動尋找直譯器，需要一點時間



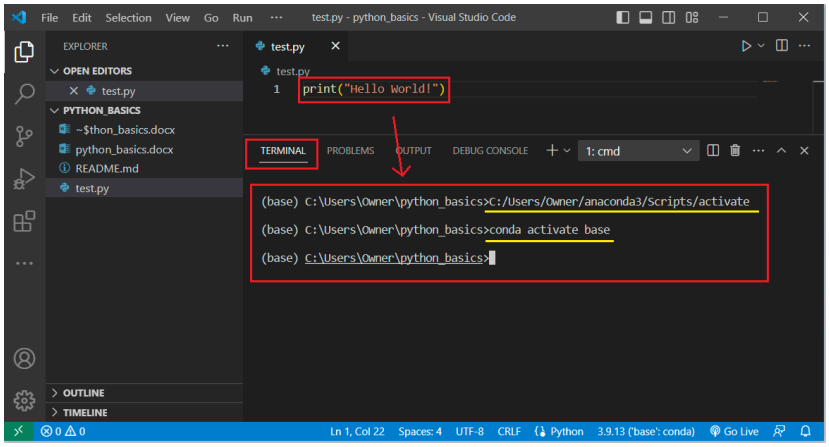
圖：在右下角看到安裝 Anaconda 的預設環境 (base)



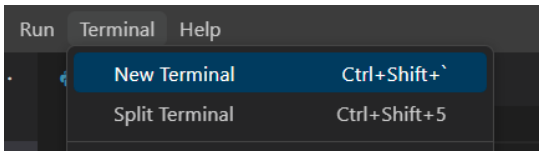
圖：按下右下角的 conda 版本圖示，可以看到安裝過的環境列表，可自由切換

建立與執行 .py 檔案

先在剛才的 test.py 檔案中，輸入「print("Hello World!")」，按下快速鍵「Ctrl + s」來儲存文字內容（程式碼）。



圖：按下快速鍵「Ctrl + ~」，顯示終端機（Terminal），並啟動 Conda 環境



圖：或是在功能列，選擇「Terminal」→「New Terminal」，效果亦同

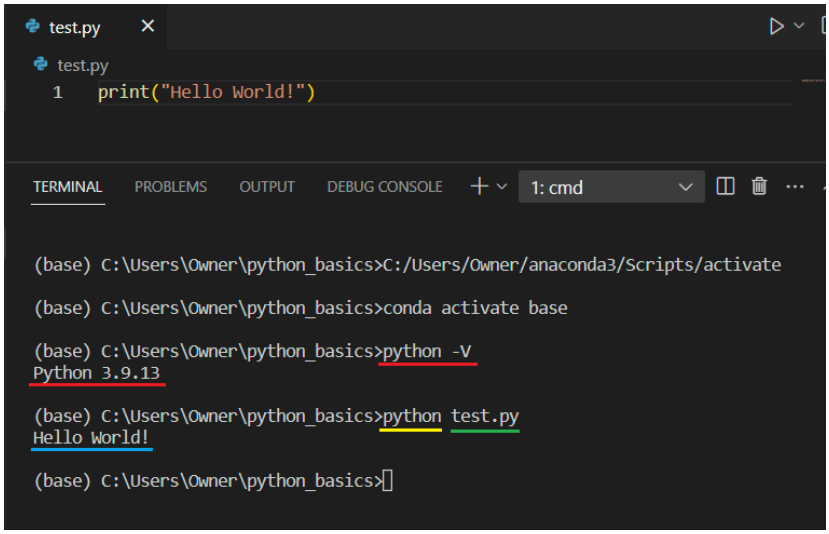


圖: 先確認 base 環境的 python 版本，然後執行 test.py 程式，出現「Hello World!」

參考資料

[1] 維基百科: Python
https://zh.wikipedia.org/zh-tw/Python
[2] How To Install Python 3 on Windows 10
https://phoenixnap.com/kb/how-to-install-python-3-windows
[3] Windwos&Mac Python 初學者為什麼選擇 Anaconda 為開發環境呢? -入門系列
https://medium.com/誤闖數據叢林的商管人 zino/windwos-mac 初學者為什麼選擇 aanconda-為開發環境呢-入門系列-f892697f1a53
[4] Conda create environment and everything you need to know to manage conda virtual environment
https://www.machinelearningplus.com/deployment/conda-create-environment-and-everything-you-need-to-know-to-manage-conda-virtual-environment/
[5] 【程式】Python 初學者的開發環境推薦
https://liah.pixnet.net/blog/post/227668100-python-初學者的開發環境推薦
[6] VS Code Python 擴充套件改進環境設定流程，並強化 IntelliSense 對單元測試的支援
https://www.ithome.com.tw/news/155413
[7] 快速上手 Anaconda 與 Jupyter 的虛擬環境與核心設定
https://medium.com/數據不止-not-only-data/anaconda-anjupyter-54144e75318f

Module 2.資料型態與變數

資料型態

資料型態	語法	說明
文字 Text	str	● 用於建立字串(string)或字元(character)。 ● 通常會使用雙引號("")或單引號('')將其前後包括起來。 ● 例如 "Hello World!"或是 'Hello World!'。 ● 一個字以上，可以視為「序列」(Sequece)
數值 Numeric	int, float	● 常見的如整數(1234)或浮點數(3.1415926)。
序列 Sequence	list, tuple, range	● list 可以一次儲存多筆資料在單一變數(變數可以想像成資料暫存的地方或是替身)，使用中括號「[]」將資料整理在一起，例如 ["人", "帥", "真", "不賴"]，或是 [9, 4, 8, 7]等。 ● tuple 跟 list 很像，只是裡面的值不能改變，使用中括號「()」將資料整理在一起，例如 ("人", "帥", "真", "好")，或是 (, 4, 8, 7)。 ● range 可以建立一個數值的列表，通常會搭配 for 迴圈語法來使用，將數值一個一個拿出來用，例如 range(10)，它會等同於 [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]。
字典 Dictionary	dict	● 以 key:value 的格式儲存資料，同時可以儲存很多 key:value 組合的資料。 ● 使用大括號「{}」將資料整理在一起。 ● 例如

		{
		"name": "Darren",
		"age": 18,
		……
		}

集合 Set	set	● 類似 list 和 tuple，但它儲存的資料「不能重複」。 ● 使用大括號「{}」將資料整理在一起。 ● 有別於 dict，它不是使用 key:value 的方式來儲存資料。 ● 例如 { "蘋果", "橘子", "水梨"}。
布林 Boolean	bool	● 用來表示真(True)或假(False)。 ● 通常用於條件判斷的結果為真或是假。
空值 None	None	● 通常用於變數初次建立的時候，一時之間不確定要給什麼值，可以先給 None。

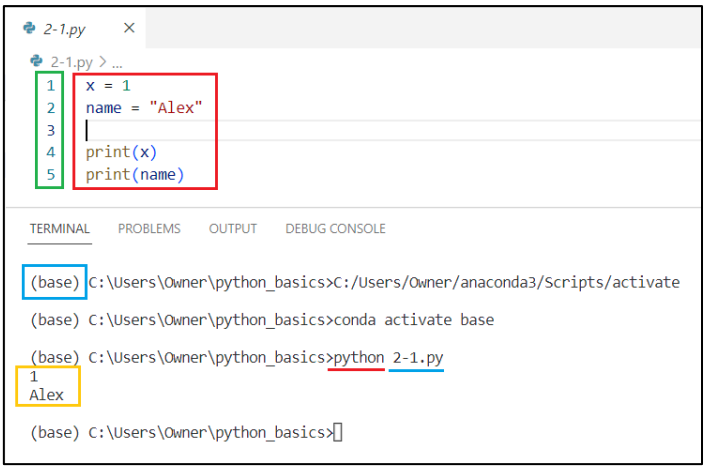
變數

變數 (variable) 就是一個暫存資料 (值) 的地方，會將資料儲存在電腦記憶體當中，原則上什麼值都給可以賦予、分配 (assign) 給它。

2-1.py

```
x = 1
name = "Alex"

print(x)
print(name)
```



圖：綠框為程式碼行號，紅框為程式碼，藍框代表 conda 環境，紅底線是指令，藍底線是檔名，黃框是程式執行結果(輸出)

變數命名規則、關鍵字與縮排

變數命名規則

- 由字母（A-z)、數字（0-9）符號和底線（_）組成。
- 必須以字母（A-z）或底線（_）開頭，不能以數字開頭。
- 區分大小寫，name、Name、NAME 是不同的變數。

```
myvar = "John"
my_var = "John"
_my_var = "John"
myVar = "John"
MYVAR = "John"
myvar2 = "John"
```

圖：正確的變數命名方式

```
2myvar = "John"
my-var = "John"
my var = "John"
```

圖：錯誤的變數命名方式

關鍵字

所謂關鍵字（keywords)，是用來保留給程式語言使用的內建語法或是變數名稱，不適合作為一般變數來使用，以下列出常見的 Python 關鍵字：

as	assert	break
class	continue	def
del	elif	else
except	False	finally
for	from	global
if	import	in
is	lambda	None
nonlocal	not	or
pass	raise	return
True	try	while
with	yield	and

縮排

Python 的縮排 (indent) 以冒號「:」來代表程式區塊 (block) 的開始，縮排通常都是使用 4 個空白鍵 (有時候會看到有人用 2 個，建議一行用 4 個)，例如：

sum = 0 score = 60	宣告 sum 變數的值為 0 宣告 score 變數的值為 60
if score >= 60: print('及格')	從 : 右方開始，到) 右方，都是程式區塊， 程式碼前放了 4 個空白 ( ) 縮排， 代表都在同一個程式區塊內。
(下一行程式) (下一行程式)	如果下一行程式前面還有縮排， 代表都在同一個程式區塊範圍內。
sum = sum + score	這是下一行程式，不受上方程式區塊的範圍影響

運算子

運算子 (operators) 就是大家常見的 + (加), - (減), * (乘), / (除), % (相除取餘數), = (賦值), > (大於), < (小於), == (等於) … 等符號，常見的有：

- 算術運算子 (Arithmetic operators)
- 賦值運算子 (Assignment operators)
- 比較運算子 (Comparison operators)

- 邏輯運算子 (Logical operators)
- 成員運算子 (Membership operators)
- 身分運算子 (Identity operators)
- 位元運算子 (Bitwise Operators)

算術運算子

運算符號	說明	範例
+	加	9 + 4 回傳 13
-	減	9 – 4 回傳 5
*	乘	9 * 4 回傳 36
/	除	9 / 4 回傳 2.25
%	相除取得餘數	9 % 4 回傳 1
**	多少的幾次方	9 ** 4 回傳 6561
//	相除取得整數	9 // 4 回傳 2

賦值運算子

運算符號	說明	範例
=	單純賦值 (值由右往左給)	a = 3 b = 4 c = a + b c 的值為 7
+=	計算加法後賦值	a = 3 b = 4 b += a 等同於 b = b + a b 的值為 7

-=	計算減法後賦值	a = 3 b = 4 b -= a 等同於 b = b - a b 的值為 1
*=	計算乘法後賦值	a = 3 b = 4 b *= a 等同於 b = b * a b 的值為 12
/=	計算除法後賦值	a = 3 b = 4 b /= a 等同於 b = b / a b 的值為 1.3333333333333333
%=	計算取餘數後賦值	a = 3 b = 4 b %= a 等同於 b = b % a b 的值為 1
**=	計算幾次方後賦值	a = 3 b = 4 b **= a 等同於 b = b ** a b 的值為 64
//=	計算取得整數後賦值	a = 3 b = 4 b //= a 等同於 b = b // a b 的值為 1

比較運算子

運算符號	說明	範例
==	等於	5 == 5 回傳 True

!=	不等於	5 != 6 回傳 True
>	大於	5 > 6 回傳 False
<	小於	5 < 6 回傳 True
>=	大於等於	5 >= 6 回傳 False
<=	小於等於	5 <= 6 回傳 True

邏輯運算子

運算符號	說明	範例
and	若是兩邊都是 True，則回傳 True	name = 'Alex' age = 18 if name == 'Alex' and age == 18: print('正確')
or	若是兩邊其中一個是 True，則回傳 True	name = 'Alex' age = 18 if name == 'Alex' or age == 16: print('正確')
not	若變數的值是 True，則回傳 False；若變數的值是 False，則回傳 True	flag = False if not(flag): print('從 False 變成 True')

成員運算子

運算符號	說明	範例
in	在序列 (sequence) 中找到指定的值，則回傳 True，反之則回傳 False。	myList = [9, 5, 2, 7] if 2 in myList: print('2 在 list 當中')
not in	在序列 (sequence)	myList = [9, 5, 2, 7]

	中沒有找到指定的值，則回傳 True，有找到則回傳 False。	`if 3 not in myList:` `  print('3 不在 list 當中')`

身分運算子

運算符號	說明	範例

is	當兩個變數是參照同一個物件(同一個記體體位置)，回傳 True。	`x = 123`
		`y = x  # y 參照 x 的記憶體位置`
		`if x is y:`
		`  print('x 和 y 是同一物件')`

is not	當兩個變數不是參照同一個物件(同一個記體體位置)，回傳 True。	`x = 123`
		`y = 456 # y 隨機佔用記憶體位置`
		`if x is not y:`
		`  print('x 和 y 不是同一物件')`

位元運算子

提示：位元運算會將運算元轉成二進制，再進行運算元之間的布林運算

運算符號	說明	範例

&	AND 運算
	(相同為 1，就是 1)
		`x = 1`
		`y = 3`
		`print( x & y ) # 輸出 1`

|	OR 運算
	(其中為 1，就是 1)
		`x = 1`
		`y = 3`
		`print( x | y ) # 輸出 3`

^	XOR 運算
	(相同為 0，相異為 1)
		`x = 1`
		`y = 3`
		`print( x ^ y ) # 輸出 2`

~	補數運算 (有正負號，運算才有義意)	`x = +3` `print(~x)# 輸出 -4`
<<	位元左移	`x = 4` `print( x << 1) # 輸出 8`

>>	位元右移	x = 4 print(x >> 2) # 輸出 1

運算子優先順序

運算子	說明
()	括號
**	冪、指數
+x -x ~x	一元運算子加號、一元運算子減號、位元運算的 not
* / // %	乘、除、除(只取商數)、相除取餘數
+ -	加、減
<< >>	位元左移、位元右移
&	位元運算 AND
^	位元運算 XOR
|	位元運算 OR
== != > >= < <= is is not in not in	比較運算子 身分運算子 成員運算子
not	(邏輯運算)True 變成 False，或是 False 變成 True
and	(邏輯運算)且
or	(邏輯運算)或

註解

註解 (comments) 在程式碼當中，通常是用於對撰寫的程式碼進行文字說明，或是希望其它行的程式碼暫時不要執行，先用單行註解或多行註解先將其關閉，待需要的時候才打開。

單行註解	多行註解	多行註解
# 單行註解	''' 多行註解 多行註解 多行註解 '''	""" 這也是多行註解 這也是多行註解 這也是多行註解 """

註：

在 VS code 當中，可以反白幾行程式碼，之後再對它們按下 `Ctrl + /`，可以作到多個單行註解；如果要解除註解，對原先註解的程式碼進行反白，再按下 `Ctrl + /`。

2-2.py

```
# 算術運算子
print(9 + 4)
print(3 / 2)
print(2 ** 3)



# 賦值運算子
a = 9
b = 4
a += b # 等於 a = a + b
print(a)
c = 3
d = 2
d **= c # 等於 d = d ** c
print(d)



# 比較運算子 (用 if 判斷)
e = 4
if e == 4:
    print('e 等於 4')
if e > 2:
    print('e 大於 2')
if e != 5:
    print('e 不等於 5')

# 如果程式區塊只需要執行 1 行程式碼，則可以這麼寫
if e == 4: print('e 等於 4')
```

```python
# 邏輯運算子
f = 8
name = 'Bill'
if f == 8 and name == 'Bill':
    print('ok')
if f == 6 or name == 'Bill':
    print('name 等於 Bill')



# 成員運算子
myList = [5, 5, 6, 6, 3, 3, 1, 2]
if 6 in myList:
    print('6 在 list 裡')
if 4 not in myList:
    print('4 不在 list 裡')
```

參考資料

[1] Python Variables
https://www.w3schools.com/python/python_variables.asp
[2] Python 運算符
https://www.runoob.com/python/python-operators.html
[3] Python Keywords
https://www.w3schools.com/python/python_ref_keywords.asp

Module 3.控制結構

循序

循序 (sequence) 指的是通常在程式語言中，程式碼是由上而下、一行一行依序執行。

選擇

選擇 (selection) 則是依條件 (condition) 讓程式分流執行。

3-1.py

```
'''
條件判斷
if ... else; if ... elif ... else; True 和 False
'''

# if
num = 10
'''
可以用以下符號來進行條件判斷
== (等於)
> (大於)
>= (大於等於)
< (小於)
<= (小於等於)
!= (不等於)
'''
if num > 5:
    print("num 大於 5")

# if else
name = 'apple'
if name == 'apple':
    print('名稱是 apple')
else:
    print("名稱不是 apple")
```

```python
# if elif else
name = 'darren'
if name == "alex":
    print("名稱: alex")
elif name == "bill":
    print("名稱: bill")
elif name == "carl":
    print("名稱: carl")
else:
    print("Not found")

# 補充: True (真) 和 False (假、偽)
is_available = True
if is_available == True:
    print("真")
else:
    print("假")

# 也可以不用加「 == True」
if is_available:
    print("真")
else:
    print("假")
```

重複

重複 (repetition) 是指當有程式區塊 (block) 需要重複執行，重複的條件為真 (True)，則該程式碼將繼續重複執行，直到條件為假 (False) 時，才會停止重複執行。這種不斷迭代 (iteration)、重複 (repeat) 執行的結構，稱為迴圈 (loop)。

3-2.py

```python
'''
while 迴圈
for 迴圈
'''
```

```python
# while 迴圈
count = 1
while count <= 5:
    print(count, end="")
    count = count + 1  # 或是寫成 count += 1



# for 迴圈 01
'''
用法
range(start, end-1)

說明
會走訪 start 到 end-1 的數字
'''
for i in range(5, 8):
    print(i, end = ",")



# for 迴圈 02
'''
用法
range(start, end-1, step)

說明
以每 step 為間距，走訪 start 到 end-1 的數字
'''
for i in range(5, 20, 2):
    print(i, end = ",")
```

在執行迴圈的過程中，若是有跳出迴圈、不繼續執行的需要，可以使用 `break` 來中斷迴圈；若是在特定條件下，不執行當前的程式區塊，或是只執行局部的程式區塊，而後立刻跳到下一個項目、下一筆資料，可以使用 `continue`。

若是在迴圈當中撰寫了條件，卻一時不確定要 break 或是 continue，抑或打算在條件中撰寫其它程式碼，暫時先保留程式區塊可用的範圍，等到想

好了再補上去，則可以使用 pass。

3-3.py

```
# break
'''
說明
當偵測到字母 t 時，就會強制結束迴圈
'''
for char in 'content':
    if char == 't':
        break
    print(char, end="")



# continue
'''
說明
當偵測到字母 t 時，
會跳過本次迴圈剩下的程式碼 print(char)，
但不會結束迴圈，仍然會進入下一圈繼續執行
'''
for char in 'content':
    if char == 't':
        continue
    print(char, end="")



# pass
'''
說明
當偵測到字母 t 時，會忽略該條件，繼續像正常迴圈一樣運行程序

備註
有時候寫 pass，是為了將某塊或某行列入 to-do
'''
for char in 'content':
```

```
    if char == 't':
        pass
    print(char, end="")
```

Module 4.字串、串列、元組、字典、集合

字串

- 「字串」(string) 如同一串字，同時由兩個以上的字元 (character) 組成。
- 字串需要用「兩個單引號」或「兩個雙引號」包起來，例如 '星期日' 或 "星期日"。
- 包住字串的兩個引號一定要一樣，不能一個單引號、一個雙引號，要同時都是單引號，或是同時都是雙引號。
- 如果包住的字只有一個，則稱之為「字元」(character)。

4-1.py

```
# 字串變數初始化
string01 = "1,2,3,4"
print(string01)


# 替換字串
'''
用法
string.replace(str1, str2)
說明
將 string 中的 str1 替換成 str2
'''
string02 = "Alex"
string02 = string02.replace('ex', 'len')
print(string02)


# 去除字串兩側空格
'''
用法
string.strip()
說明
去除字串 string 左、右兩邊的空格
'''
```

```
string03 = "        ___ccc___        "
print(string03)
print(string03.strip())


# 字串變成小寫或大寫
'''
用法
string.lower()
說明
將字串 string 裡的字母全部改成小寫
'''
print("CAR".lower())

'''
用法
string.upper()
說明
將字串 string 裡的字母全部改成大寫
'''
print("good".upper())
```

補充：字串格式化

輸出結果的時候，若是需要嵌入必要的文字，建立字串樣版 (string template)，或是嘗試將文字進行排列與調整，可以將字串格式化。

使用 %

語法	說明
%s	以 str() 函式輸出文字
%f	以 浮點數 輸出文字
%d	以 十進制 輸出文字
%%	輸出百分比 (類似跳脫字元)

```
4-1-1.py
```

```
'''
格式化字串 - 使用 %
'''
# 多組文字
msg = '%s, %s!' % ('Hello', 'World')
print(msg)

# 整數
msg = 'I am %d years old.' % 5
print(msg)

# 文字與整數
msg = '%s is %d years old.' % ('Alex', 18)
print(msg)

# 指定寬度 (維持 10 個字元長度，預設向右對齊)
msg = '[%10s]' % 'Hello'
print(msg)

# 靠左對齊 (維持 10 個字元長度，向左對齊)
msg = '[%-10s]' % 'Hello'
print(msg)

# 指定浮點數位數
msg = '[%8.3f]' % 12.3456
print(msg)

# 指定文字長度上限 (只有文字，才在格式化字串中加入「.」來限定字串長度)
msg = '[%.3s]' % 'Hello'
print(msg)

# 空白補 0
msg = '[%06.2f]' % 3.1415926
print(msg)
```

使用 .format()

4-1-2.py

```
'''
格式化字串 - 使用 string.format()
'''
# 嵌入文字
msg = '{}, {}!'.format('Hello', 'World')
print(msg)

# 改變參數順序
msg = '{1}, {0}'.format('Hello', 'World')
print(msg)

# 指定寬度
msg = '[{:10}]'.format('Hello')
print(msg)

# 靠右對齊
msg = '[{:>10}]'.format('Hello')
print(msg)

# 靠左對齊
msg = '[{:<10}]'.format('Hello')
print(msg)

# 置中對齊
msg = '[{:^10}]'.format('Hello')
print(msg)

# 指定浮點數位數
msg = '[{:8.3f}]'.format(12.3456)
print(msg)

# 指定文字長度上限 (只有文字，才在格式化字串中加入「.」來限定字串長度)
msg = '[{:.3}]'.format('Hello')
```

```
print(msg)

# 空白補 0
msg = '[{:06.2f}]'.format(3.1415926)
print(msg)
```

使用 f-string (Python 版本 > 3.6)

4-1-3.py

```
# f-string (又稱作 formatted string literals, version >= 3.6)
name = "Darren"
age = 18
result = f"name: {name}, age: {age}"
print(result)

# 靠左對齊、靠右對齊
result = f"name: [{name:<10}], age: [{age:>10}]"
print(result)

# 靠左對齊、靠右對齊，字數不足，補「厂」(沒寫補什麼，預設空格)
result = f"name: [{name:厂<10}], age: [{age:厂>10}]"
print(result)

# 置中對齊
result = f"name: [{name:^10}], age: [{age:^10}]"
print(result)

# 指定文字長度上限 (只有文字，才在格式化字串中加入「.」來限定字串長度)
result = f"name: [{name:^10.2}]"
print(result)
```

補充：input() 與 eval()

若是希望能在程式執行期間，讓使用者可以自行輸入資料 (類似提示對話視窗 prompt)，之後整合到程式碼之後輸出結果，可以使用 input() 函式。另一個是 eval()，可以用來執行動態生成、執行 Python 程式碼，有人透過該特

性來自動判斷輸入字串為整數或浮點數，不過透過字串動態執行程式碼，會有安全性的問題 (透過輸入字串來執行惡意程式碼)，請謹慎使用。

4-1-4.py

```
# 取得使用者輸入的資料
data = input('輸入文字後，按下 Enter: ')
print(data)

# 將使用者輸入強制轉型成 int
num = int(input('請輸入數字: '))
print(type(num))
print(num)
```

4-1-5.py

```
# 簡單計算
expression = "2 + 3 * 4"
result = eval(expression)
print(result)


# 執行程式碼
x = 10
code = "x * 2 + 5"
result = eval(code)
print(result)


# 字串轉數值
result = eval("3.14")
print(type(result))
result = eval("314")
print(type(result))
```

串列

- 「串列」(list) 或稱為「列表」，跟其它程式語言當中的「陣列」(array) 很類似。
- 使用「中括號」[ ] 將資料儲存起來。

- 可以想像成一個放置藥丸的盒子，從星期天開始。
- 例如【'星期天', '星期一', '星期二', '星期三', '星期四', '星期五', '星期六'】。
- 這個藥盒子內，有各自存取的代號(位置)，例如 0 代表「星期天」，1 代表「星期一」，2 代表「星期二」，依此類推。
- 這個存取的代號 0、1、2…等等，稱之為「索引」(index)。
- 可以儲存各種資料型態

4-2.py

```
# 初始化一個 list
ids = [1, 2, 3, 4, 5, 6]
print(ids)


# 新增元素在 list 尾端
ids.append(7)
print(ids)


# 修改索引位置的元素
ids[4] = 99
print(ids)


# 刪除索引位置的元素
ids.pop(4)
print(ids)


# 新增元素 9 在指定索引 1，其餘元素往後移
ids.insert(1, 9)
print(ids)


# 移除指定的值
ids.remove(9)
print(ids)
```

```
# 補充: 字串分割成 list
'''
用法
string.split()
說明
默認以空格、換行字元分割字串 string，回傳 list
'''
string04 = "1,2,3,4"
list04 = string04.split(',')
print(list04)


# 補充: 將 list 元素合併成字串
'''
用法
string.join(seq)
說明
以 string 為分隔符號，將 seq 中的元素串起來成為一個新的字串
'''
list05 = ['古道', '西風', '瘦馬']
string05 = '-'.join(list05)
print(string05)

string05 = ''.join(list05)
print(string05)
```

補充：序列物件分割（Slicing）

- 序列物件包含 String、List 及 Tuple 等，我們可透過其順序取得元素
- 中括號為常見物件的索引值取值（Indexing）或分割之語法
  - 索引值取值指的是取出一個值
  - 分割指的是取出一部分值

索引值：	0	1	2	3	4	5	6	7	8	9	10	11
	'H'	'e'	'l'	'l'	'o'	','	'W'	'o'	'r'	'l'	'd'	'!'
索引值：	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

圖: 把 sequence 當成 list 來存取

4-3.py

```
# 初始化一個字串
myStr = 'Hello,World!'


'''
透過冒號（:）進行分割
'''
# 從頭取到 5（不包含第 5 個元素，實際為 0 ~ 4）
print( myStr[0:5] ) # 索引從 0 開始，到 5 - 1 的索引位置

# 從頭取到 5（不包含第 5 個元素，實際為 0 ~ 4）
print( myStr[:5] ) # 冒號前面留空，效果跟 [0:5] 一樣

# 從 7 取到尾
print( myStr[7:] )  # 冒號後面留空

# 從 7 取到 9（不包含第 9 個元素，實際為 7 ~ 8）
print( myStr[7:9] )



'''
使用負號
'''
# 從倒數第 10 取到倒數第 7（不包含倒數第 7 的元素）
print( myStr[-10:-7] )

# 從倒數第 5 取到尾
print( myStr[-5:] )

# 從頭取到倒數第 7（不包含倒數第 7 的元素）
print( myStr[:-7] )


# 取得檔案全名
```

```
string06 = "/home/darren/ebook.txt"
list06 = string06.split("/")
print(list06[-1])


# 搜尋字串
string07 = 'believe'
result07 = string07.find('lie')
'''
用法
string.find(str)
說明
回傳 string 第一次在字串 string 中出現的 index，若找不到則回傳 -1
'''
print(result07)
```

補充：List Comprehension

使用一行程式碼，搭配 for 迴圈，就可以產生一個 list。

4-3-1.py	
一般 list 用法	List Comprehension

`list01 = []`
`for x in range(10):`
`  list01.append(x)`
`print(list01)`
	`# 一行直接建立 list`
	`list02 = [x for x in range(10)]`
	`print(list02)`
	
	`# 還可以設定條件`
	`list03 = [x for x in range(10) if x % 2 == 0]`
	`print(list03)`

補充：enumerate()

使用 for 迴圈的時候，若是需要同時回傳序列物件的索引 (index) 與對應的值 (value)，可以使用 enumerate 函式。

4-3-2.py
`'''透過 enumerate() 逐一將序列的索引與值進行回傳'''` `seq = ['人', '帥', '真好']`

```
for index, value in enumerate(seq):
    print("=" * 30) # 將字元或字串進行乘法運算，可以作到字元/字串生成的效果
    print(index) # 輸出 索引
    print(value) # 輸出 值
```

元組

- 元組 (tuple) 。
- 有序號、索引概念。
- 允許重覆的值。
- 跟 list 很類似，只是不可更改元組的資料。
- 可以使用 for 迴圈讀出一筆筆的資料。

4-4.py

```
# Tuple 初始化: 第一種
myTuple01 = ("人", "帥", "得體")
print(myTuple01)
print(type(myTuple01))

# Tuple 初始化: 第二種
myTuple02 = "哆", "啦", "A", "夢"
print(myTuple02)
print(type(myTuple02))

# 透過指定索引輸出值
print( myTuple01[1] )
print( myTuple02[1] )

# 複數變數修改值
a = 10
b = 20
print("交換前: a = {}, b = {}".format(a, b))
a, b = b, a
print("交換後: a = {}, b = {}".format(a, b))

# list 可以修改指定索引的值
myList = ["人", "帥", "任性"]
```

```
myList[2] = "真好"
print(myList)

# tuple 不可以修改指定索引的值
myTuple = ("人", "帥", "任性")
myTuple[2] = "真好"
print(myTuple)

# 新增資料到 tuple (類似合併的方式)
myTuple01 = myTuple01 + ("哈",) # 只加入 1 個字
print(myTuple01)
myTuple01 = myTuple01 + ("哈", "哈") # 2 個字以上，最後一個元素不用加
「,」
print(myTuple01)

# tuple 之間可用 + 來合併
myTuple03 = myTuple01 + myTuple02
print(myTuple03)

# 用 for 迴圈逐一輸出資料
for value in myTuple01:
    print(value)

# 用 len 計算 tuple 資料個數
print( len(myTuple02) )

# 因為無法修改 tuple 的資料，所以也無法指定索引進行刪除
del myTuple01[1]
print(myTuple01)

# 只能從記憶體刪除整個 tuple 變數
del myTuple01
print(myTuple01)
```

字典

- 「字典」(dict)，跟其它程式語言的「物件」(Object) 很類似。

- 透過大括號 { } 來存放資料。
- 結構以 key:value 來儲存資料，key 可以想像成一個變數。
- 一個字典可以放置多組 key:value，每組之間用逗號 (,) 分隔。
- 例如 {'name': 'Darren', 'age': 18, 'height': 172, 'weight': 100}
- 其中 'name' 就是字典其中的 key，而 'Darren' 就是此 key 對應的 value。
- 與 List 不同的是，Dict 通常是透過 key 來存取 value，沒有特定的 index。

4-5.py

```
# 初始化 dict
dict01 = {"蘋果": 100, "橘子": 20, "水梨": 50}
print(dict01)

'''
上面的例子，可以透過排版，寫成:

dict01 = {
    "蘋果": 100,
    "橘子": 20,
    "水梨": 50
}
'''

# 上面的初始化過程，也可以寫成:
dict01 = dict()
dict01['蘋果'] = 100
dict01['橘子'] = 20
dict01['水梨'] = 50
print(dict01)

# 印出蘋果的價格
print(dict01["蘋果"])

# 修改橘子的價格
dict01["橘子"] = 30
```

```
print(dict01["橘子"])

# 刪除 水梨
del dict01["水梨"]
print(dict01)


'''
有時候會遇到比較複雜的結構，需要一層一層去走訪，取得資料
'''
dict02 = {
    'name': 'Darren',
    'age': 18,
    'info': {
        'nickname': 'boatman',
        'favorite_role': 'Doraemon',
        'phone_number': [
            '0911111111',
            '0922222222',
            '0933333333'
        ]
    }
}

# 取得 nickname
print( dict02['info']['nickname'] )

# 取得所有 phone_number
for number in dict02['info']['phone_number']:
    print(number)
```

補充：Dictionary Comprehension

使用一行程式碼，搭配 for 迴圈，就可以產生一個 dict。

```
4-5-1.py
```

```python
# 用來建立 dict key(s) 的 list
list01 = ["name", "school", "phone_number"]

# 建立 dict
dict01 = {key: "" for key in list01}

# 檢索目前的 dict
print(dict01)

# 各別填寫 dict 的 value
dict01['name'] = 'Alex'
dict01['school'] = 'ntu'
dict01['phone_number'] = '0911111111'

# 檢索結果
print(dict01)
```

補充：dict.items()

將 dict 的 key 和 value 組合，透過 for 迴圈，將 dict 當中所有的 key-value 組合，以 tuple 的格式回傳。其中 tuple 的索引 [0] 為 key，tuple 的索引 [1] 為 value。

4-5-2.py

```python
# 建立字典
dict02 = {"name": "John", "age": 25, "city": "New York"}

# 透過 items()方法取得字典的鍵值對
for t in dict02.items():
    print(t)
    print(f"key: {t[0]}, value: {t[1]}")
    print("=" * 50)
```

集合

- 無序號、索引概念
- 無索引(隨機出現)
- 沒有重覆(一樣的只會出現一次)

4-6.py

```
# Set 初始化: 第一種
mySet01 = {"網路", "爬蟲", "真好玩"}
print(mySet01)

# Set 初始化: 第二種 (裡面放 tuple)
mySet02 = set( ("網路", "爬蟲", "真好玩") )
print(mySet02)

# 一筆筆讀資料 : for 迴圈
for data in mySet02:
    print(data)

# 總共有幾筆資料: len()
print( len(mySet02) )

# 因為沒有序號、索引的概念，所以無法透過指定索引輸出
print( mySet01[0] )

# 添加一筆資料: add()
mySet01.add("嗎?")
mySet02.add("對不對?")

print(mySet01)
print(mySet02)

# 此時新增重複的資料，不會增加
mySet01.add("真好玩")
print(mySet01)

# 添加多筆資料: update()
mySet01.update(["甲", "乙", "丙"])
mySet02.update(["子", "丑", "寅"])
print(mySet01)
print(mySet02)
```

```python
# 查詢 set 中有沒有我要的資料: in (只回傳 true 或 false)
if "乙" in mySet01:
    print("有資料")
else:
    print("找不到資料")

# 刪除元素: discard() or remove()
'''如果 .discard() 的元素不存在，不會報錯'''
mySet01.discard("丙")
print(mySet01)
'''如果 .remove() 的元素不存在，會報錯 (KeyError)'''
mySet02.remove("丑")
print(mySet02)

# 清空: clear()、del
'''.clear() 會清空 set 變數，但變數依然存在，所以會印出空 set'''
mySet01.clear()
print(mySet01)
''' del 會將 set 變數從記憶體中刪除，所以刪除完後，變數會變成未宣告的狀態
'''
del mySet02
print(mySet02)
```

參考資料

[1] Python 基礎教學
https://www.runoob.com/python/python-tutorial.html
[2] String Formatting in Python
https://www.scaler.com/topics/python/string-formatting-in-python/
[3] Python 字串格式化教學與範例
https://officeguide.cc/python-string-formatters-tutorial/
[4] 如何使用 Python 進行字串格式化
https://blog.techbridge.cc/2019/05/03/how-to-use-python-string-format-method/
[5] Python String Formatting
https://www.w3schools.com/python/python_string_formatting.asp

[6] List Comprehension: Python 的 For Loop 怎樣使用？
https://pythonviz.com/basic/list-comprehension/
[7] Python3 enumerate() 函数
https://www.runoob.com/python3/python3-func-enumerate.html
[8] Python3 字典 items() 方法
https://www.runoob.com/python3/python3-att-dictionary-items.html

Module 5.函式

函式 (function) 將經常使用的程式碼打包起來，變成一種模組，每執行一次函式，相當於把模組的程式碼全部執行過，透過 def 關鍵字來定義。

基本用法

```
# 函式的定義方式
def 函式名稱():
    陳述句(statement，代表程式要執行的動作)

# 執行函式，直接使用函式名稱，後面加上()
函式名稱()
```

變數傳遞

回傳 值(結果)	傳遞變數

# 傳遞在函式當中執行的結果 	
def 函式名稱():
return 您的執行結果
 	
# 函式執行完畢後，會將結果回傳給變數 	
變數 = 函式名稱()
	# 傳遞在函式當中執行的結果
	def 函式名稱(參數[,參數 2,…]):
	return 整合變數後的結果
	
	# 函式執行完畢後，會將結果回傳給變數
	變數 = 函式名稱(引數[,引數 2,…])

5-1.py

```
'''不回傳 值'''
# 定義函式
def show_name():
    print("Doraemon")

# 執行函式
show_name()



'''回傳 值'''
```

```python
# 定義函式
def get_name():
    name = "Doraemon"
    return name
    # 也可以寫成 return "Doraemon"

# 執行函式
result = get_name()

# 輸出回傳結果
print(result)



'''傳遞參數'''

# 定義函式
def get_greeting(name):
    return "Hello, " + name

# 執行函式
result = get_greeting("Darren")

# 輸出回傳結果
print(result)
```

區域變數與全域變數

- 當程式區塊 (block) 內外都有相同的變數名稱：
  - 程式區塊內部的變數，叫作區域變數 (local variable)。
  - 程式區塊外部的變數，會稱為外部變數。
  - 如果變數在程式碼的最外層，沒有被任何程式區塊 (block) 包起來，該變數叫作全域變數 (global variable)。
- 想在函式內部使用、修改外部變數的值，必須在變數前加上 global 關鍵字。
- 不能在加上 global 的時候進行變數初始化。

區域變數無法修改全域變數	區域變數可以修改全域變數

```python
# 全域變數
name = 'Bill'

# 定義函式
def 函式名稱():
    # 區域變數
    name = 'Doraemon'

# 執行函式
函式名稱()

# 輸出 name
print(name) # 輸出 Bill
```

```python
# 全域變數
name = 'Bill'

# 定義函式
def 函式名稱():
    # 使用 global 關鍵字，
      讓函式可以取存全域變數的值
    global name

    # 區域變數
    name = 'Doraemon'

# 執行函式
函式名稱()

# 輸出 name
print(name) # 輸出 Doraemon
```

5-2.py

```python
'''區域變數無法修改全域變數'''
# 全域變數
name = "Bill"

# 定義函式
def set_name():
    # 區域變數
    name = "Doraemon"

# 執行函式
set_name()

# 輸出結果
print(name)



'''區域變數可以修改全域變數'''
```

```python
# 全域變數
name = "Bill"

# 定義函式
def set_name():
    '''
    想在函式內部使用、修改外部變數的值，
    必須在變數前加上 global 關鍵字，
    但是不能在加上 global 的時候進行變數初始化
    '''
    global name
    name = "Doraemon"

# 執行函式
set_name()

# 輸出結果
print(name)
```

匿名函式 Lambda

所謂匿名函式 (anonymous function)，許多程式語言都有支援，在程式語言 Python 當中叫作 Lambda，只需要撰寫一行的程式碼，就有一般函式的效果，相對其它函式定義方法來得簡潔。

將變數作為函式名稱 = lambda 參數 1[,參數 2,…] : 執行回傳結果

一般函式	Lambda

`def add(x, y):`
`    return x * y`
 	
`print( add(3, 5) )`
	`add = lambda x, y : x * y`
	
	`print( add(3, 5) )`

`def abs(x):`
`    if x >= 0:`
`        return x`
`    else:`
`        return -x`
	`abs = lambda x : x if x >= 0 else -x`

print(abs(5)) print(abs(-5))	print(abs(5)) print(abs(-5))
def product(x, y): return x * y print(product(4, 5))	print((lambda x,y : x * y)(4, 5))

Lambda 通常會與一次性影響所有序列資料的函式整合使用，例如 map 或 sorted 等。

map	sorted

# 與 map 函式一起使用 	
myList = list(map(lambda x : x ** 2, [1, 2, 3, 4, 5]))
print(myList)
	# 與 sorted 函式一起使用
	myList = [
	    {"name": "Alex", "age": 18},
	    {"name": "Bill", "age": 25},
	    {"name": "Carl", "age": 21},
	    {"name": "Darren", "age": 10},
	]
	list_sorted = sorted(
	    myList,
	    key=lambda d: d['age'],
	    reverse=False
	)
	print(list_sorted)

5-3.py

# 帶入參數，並將計算結果直接回傳 	
add = lambda x, y : x * y
print(add(3, 5))
 	
# 回傳之前的條件判斷 (需要練習一下語法，否則一開始不好理解) 	
abs = lambda x : x if x >= 0 else -x
print(abs(5))
print(abs(-5))
 	
# 不使用變數當作函式名稱，直接執行 	

```python
print( (lambda x,y : x * y)(4, 5) )

# 與 map 函式一起使用
list_map = list(map(lambda x : x ** 2, [1, 2, 3, 4, 5]))
print(list_map)

# 與 sorted 函式一起使用
myList = [
    {"name": "Alex", "age": 18},
    {"name": "Bill", "age": 25},
    {"name": "Carl", "age": 21},
    {"name": "Darren", "age": 10},
]
list_sorted = sorted(myList, key=lambda d: d['age'], reverse=False)
print(list_sorted)
```

雖然匿名函式有它的便利性，若是撰寫的程式邏輯相當複雜，還是比較適合使用一般函式，比較容易維護。

參考資料

[1] [Python 教學]5 個必知的 Python Function 觀念整理
https://www.learncodewithmike.com/2019/12/python-function.html
[2] Wiki - 匿名函式
https://zh.wikipedia.org/zh-tw/匿名函数
[3] Python- lambda 函式
https://medium.com/seaniap/python-lambda-函式-7e86a56f1996
[4] Python3 map() 函数
https://www.runoob.com/python3/python3-func-map.html
[5] Python3 sorted() 函数
https://www.runoob.com/python3/python3-func-sorted.html

Module 6.物件導向簡介

基本認識物件導向

物件導向程式設計 (Object-Oriented Programming) 是幾乎把所有在 Python 中的東西都看成物件 (object)，擁有它自己的屬性 (properties) 和方法 (methods)。

例如車子有自己的烤漆顏色、4 門或 5 門、車身的寬高、安全氣囊數量等等，這些是屬性；車子可以按下喇叭、煞車、轉彎、加速等，都是它的方法。例如人類有自己的身高、體重、性別、髮色、膚色、牙齒顆數、是否有戴眼鏡等，這些是屬性；人類可以跑、跳、說話、睡覺等，這些是方法。

任何東西都擁有自己的屬性和方法，將這樣的概念用在程式設計上，就叫作物件導向程式設計。

類別

類別 (class) 就像是一個樣式的名稱，裡面已經事先定義好基本的、通用的屬性和方法，例如人類通常會有兩隻手、兩條腿，同時也會爬行、走路、跑、跳等。

```
class 類別名稱:
    (縮排後，撰寫程式碼)
```

方法

方法 (methods) 跟函式 (functions) 的感覺很像，都是執行一連串在程式區塊中定義好的程式碼 (code)，在每一個類別 (class) 中，都會有一些 Python 內建且需要在一開始定義的函式。

在建立類別的時候，常常會看到 __init__(self)，指的是建立物件 (或稱為實體) 的時候，預設第一時間會執行的方法，有時會稱為建構子 (constructor)。例如人類剛出生的時候，通常一開始就會放聲大哭，很可能是在 __init__(self) 裡面，定義了放聲大哭的方法。

```
class 類別名稱:
    def __init__(self):
        print('Python')
    def 方法1():
```

```
        return 'Hello World!'
    def 方法 2():
        return 'Good job!'

物件(實體) = 類別名稱() # 建立物件(又稱實體化)時，直接輸出 Python
print( 物件.方法 1() ) # 輸出 Hello World!
print( 物件.方法 2() ) # 輸出 Good job!
```

6-1.py

```
# 建立一個家長類別
class Parent():
    def __init__(self):
        pass
    def post(self):
        return '發話中'

# 實體化 (instantiation)
obj_parent = Parent()
print(obj_parent.post())
```

屬性

屬性 (properties) 可以想像成類別內部的變數，self 是讓物件或是類別本身可以存取在類別當中定義好的屬性和方法。

```
class 類別名稱:
    def __init__(self, 參數 1, 參數 2)L
        self.內部變數 1 = 參數 1
        self.內部變數 2 = 參數 2
        self.內部變數 3 = 自訂值

物件(實體) = 類別名稱('Hello', 'World')
print(物件.內部變數 1) # 輸出 Hello
print(物件.內部變數 2) # 輸出 World
print(物件.內部變數 3) # 輸出 在程式中自訂的值
```

6-2.py

```
# 建立一個家長類別
class Parent():
    def __init__(self, height, weight):
        self.height = height
        self.weight = weight
        self.money = 100000000
    def post(self):
        return '發話中'

# 實體化 (instantiation)
obj_parent = Parent(165, 80)
print(obj_parent.height) # 輸出 165
print(obj_parent.weight) # 輸出 80
print(obj_parent.money) # 輸出 10000000
print(obj_parent.post())# 輸出 發話中
```

繼承

繼承 (Inheritance) 是建立一個新的類別 (class) 時，把前一個定義好的類別內容拿過來使用，包括前一個類別的屬性和方法，同時也可以在新類別當中定義新的屬性和方法。

例如：

```
class Parent():
    def __init__(self):
        self.height = 172
        self.weight = 95
    def post(self):
        return '發話中'

class Child(Parent):
    def __init__(self): # 這裡的 __init__ 屬於 Child 類別的
        super().__init__() # 這裡將 Parent 所有屬性、方法繼承下來
    def reply(self):
        return '回話中'
    def content(self):
        return '我知道了'
```

6-3.py

```python
# 建立家長類別
class Parent():
    def __init__(self, height, weight):
        self.height = height
        self.weight = weight
        self.money = 100000000
    def post(self):
        return '發話中'

# 建立子類別
class Child(Parent):
    '''
    這裡的 __init__ 屬於 Child 類別的，會將 Parent 的 __init__ 覆蓋掉
    '''
    def __init__(self, height, weight):
        '''
        super() 代表父類別，
        super().__init__() 代表使用父類別的建構子，
        在執行這行程式碼時，會自動將 Parent 所有屬性、方法繼承下來
        (包括 self.money)
        '''
        super().__init__(height, weight)
        self.height = height
        self.weight = weight
    def reply(self):
        return '回話中'
    def content(self):
        return '我知道了'
    def post(self): # overwrite (覆寫) 父類別的方法
        return '不要再發話了'

# 實體化 Parent 物件
obj_parent = Parent(165, 80)
print(obj_parent.height)
print(obj_parent.weight)
```

```
print(obj_parent.post())

# 實體化 Child 物件
# (此時 Child 除了自己的屬性、方法外，還擁有父類別的屬性、方法)
obj_child = Child(172, 95)
print(obj_child.height)
print(obj_child.weight)
print(obj_child.money)
print(obj_child.reply())
print(obj_child.content())
print(obj_child.post())
```

參考資料

[1] Python Inheritance
https://www.w3schools.com/python/python_inheritance.asp
[2] Python 面向对象 (物件導向)
https://www.runoob.com/python/python-object.html

Module 7.例外處理

認識例外處理

例外 (Exception) 就是程式執行過程中，發生錯誤，影響程式的正常執行，所立刻執行的事件 (event)。

通常程式執行都會被中斷，偶爾也會顯示一些造成問題的原因；也可以自訂例外發生時的程式碼，不見得一定要使用預設的警示結果。以下是個人常見的事件類別：

事件類別	說明
BaseException	基本例外事件類別
Exception	最常用的例外事件類別
IOError	輸入/輸出發生錯誤的事件類別
NameError	變數尚未宣告所發生的事件類別
IndexError	序列中沒有這個索引(index)的事件類別
RuntimeError	執行過程中發生錯誤的事件類別
SyntaxError	Python 語法錯誤的事件類別
KeyboardInterrupt	使用 Ctrl + C 取消程式執行的事件類別
IndentationError	縮排錯誤的事件類別
Warning	顯示警告訊息的事件類別 (通常不會中斷程式)
DeprecationWarning	功能、指令、語法等不再被支援的事件類別
RuntimeWarning	可能造成程式執行錯誤的事件類別
SyntaxWarning	程式碼語法可能有問題的事件類別

try-except 陳述

例外的處理，可以使用「try: … except: …」語法，try: 是預設的程式區塊，當 try: 程式區塊的程式碼在執行期間發生錯誤，會直接中斷，跳到 except: 程式區塊來繼續執行。

try:

預設正常執行程式的程式區塊

except 例外事件類別名稱 1:

拋出例外時，符合例外事件類別名稱 1，則在這個程式區塊繼續執行

except 例外事件類別名稱 2:

拋出例外時，符合例外事件類別名稱 2，則在這個程式區塊繼續執行

else:

沒有例外發生的程式區塊

finally:

無論是否正常執行，最後都會執行這裡的程式區塊

7-1.py

```
# 可以連續定義數個例外處理機制，
# 最符合的例外事件類別會先拋出
try:
    print(x)
except NameError:
    print("x 尚未宣告")
except:
    print("某個東西出錯了")



# 多層例外處理: 內部的例外處理會優先補捉，
# 捕捉不到，才會由在外部的例外處理來補捉
try:
    try:
        print(y)
    except NameError as err:
        print("NameError:")
        print(str(err))
except Exception as err:
    print("Exception:")
    print(str(err))



# 如果沒有錯誤，就會執行 else 程式區塊
# finally 程式區塊是無論有無錯誤，都會執行
try:
    z = 5
    print(z)
except NameError:
```

```
    print("變數尚未宣告")
else:
    print('沒有例外發生')
finally:
    print('程式結束')
```

例外處理 raise 陳述

如果有自訂例外的需求，可以使用 raise 語法來拋出例外訊息。

7-2.py

```
# 自訂例外處理
x = -1

try:
    if x < 0:
        raise Exception("數字小於 0")
except Exception as e:
    # 可以在這裡加入處理錯誤的程式碼，例如 logging 機制
    print(e)


# 自訂型別錯誤的例外處理
x = "hello"

try:
    if type(x) is not int:
        raise TypeError("只接受整數型別")
except TypeError as e:
    # 自訂處理錯誤的程式碼
    print(e)
```

補充：logging

有別於單純的 print，logging 模組提供了一套全面且靈活的日誌記錄系統。以下是它的主要特性以及常用功能的說明：

- 主要特性：
  - 多等級的日誌記錄：

支援 DEBUG、INFO、WARNING、ERROR 和 CRITICAL 五種日誌等級；只有比設定級別（包含）還嚴重的 log 會被記錄。

- 高度的可配置性：
  可以為不同的日誌處理器（handlers）設定不同的格式、過濾條件和日誌層級。
- 模組化：
  可以為不同模組或應用程式設置獨立的 logger。
- 支援多種輸出：
  內建了多種 handlers，包括將日誌輸出到終端機、文件、HTTP 伺服器等。
- Thread-safe：
  在多執行緒環境下，日誌操作是安全的。

- 常用功能：
  - Logger:
    這是日誌的主體，通常你會根據需要，為每個模組或應用程式取得或建立一個 Logger 實體。
  - Handler:
    負責控制日誌訊息的輸出，例如輸出到文件或者輸出到終端機。常見的 handlers 有 StreamHandler 和 FileHandler。
  - Formatter:
    決定日誌訊息的最終輸出格式。
  - Filter:
    可以確定要輸出哪些日誌記錄。

以下是每個等級的說明以及適用的時機點：

等級	說明
DEBUG	描述: 用於細節的診斷資訊，通常僅在測試時使用。 適用時機: 測試特定問題時需要的詳細資訊，例如變數的值、進入特定程式碼分支的次數等。
INFO	描述: 用於一般的操作訊息，確認事情按預期運行。 適用時機: 讓你知道系統的運行狀態或進度，例如"系統啟動完成"、"成功

	連接到資料庫"或"成功處理了 X 個請求"。
WARNING	描述: 表示一個潛在的問題或將來可能會成為問題的事情。 適用時機: 當某些事情不如預期但不至於造成系統停擺的情況，例如某些資源使用達到一定門檻、即將過期的憑證或已過時的 API 使用。
ERROR	描述: 表示嚴重的問題，部分功能可能不能正常運作。 適用時機: 當系統遇到錯誤且某些功能不能正常運作時，例如無法連接到資料庫、無法讀取必要的設定文件或某些請求處理失敗。
CRITICAL	描述: 表示非常嚴重的錯誤，可能會導致整個應用程式或系統的崩潰。 適用時機: 當出現必須立即關注和解決的重大問題，例如記憶體不足、核心系統組件失敗或配置錯誤可能導致整個系統無法運作。

7-3.py

```
import logging

'''
Logging 設定
'''
# 基本設定
logger = logging.getLogger('my_app')

# 設定等級
logger.setLevel(logging.INFO)

# 設定輸出格式
formatter = logging.Formatter('%(asctime)s - %(levelname)s -
%(message)s', "%Y-%m-%d %H:%M:%S")

# 儲存在 log 當中的事件處理器
```

```python
fileHandler = logging.FileHandler('my_app.log', mode='a',
encoding='utf-8') # a: append, w: write
fileHandler.setFormatter(formatter)

# 輸出在終端機介面上的事件處理器
console_handler = logging.StreamHandler()
console_handler.setFormatter(formatter)

# 加入事件
logger.addHandler(console_handler)
logger.addHandler(fileHandler)


# 基本範例
for x in range(0, 11):
    if x > 5:
        logger.warning(f"數字 {x} 大於 5")
    else:
        logger.info(f"數字 {x} 在合理範例")

# 整合 try - except 機制 (非正式)
for x in range(0, 11):
    try:
        if x > 5:
            raise Exception(f"數字 {x} 大於 5")
        else:
            logger.info(f"數字 {x} 在合理範例")
    except Exception as err:
        logger.warning(err)
```

正確使用這些日誌等級可以幫助開發者和系統管理員更容易辨識和解決問題，並了解系統的運行狀態。

參考資料

[1] Python 异常处理 (例外處理)
https://www.runoob.com/python/python-exceptions.html
[2] Python Try Except

https://www.w3schools.com/python/python_try_except.asp
[3] Python 中的 Log 利器：使用 logging 模組來整理 print 訊息
https://zhung.com.tw/article/python中的log利器-使用logging模組來整理print訊息/

Module 8. 檔案處理

open() 函式

可以使用內建的 open() 函式來進行檔案處理，其中包含了 4 種檔案模式：

- "r": Read (讀取檔案)
- "a": Append (附加資料到檔案內容的最尾端或最後一行)
- "w": Write (寫入檔案)
- "x": Create (建立檔案)
- "t": Text (文字模式)
- "b": Binary (二進制模式，例如用於 images 的讀取)

讀取檔案

8-1.py

```
# 讀取檔案內容 - 舊的寫法
f = open(file="./2-1.py", mode="r", encoding="utf-8")
print( f.read() )
f.close() # 關閉檔案，否則執行期間，在關閉前，會變成唯讀狀態

# 印出 50 個 等號
print("=" * 50)

# 讀取檔案內容 - 現今常用的寫法 (執行完畢會自動關閉檔案)
with open(file="./2-1.py", mode="r", encoding="utf-8") as f:
    print( f.read() )

# 印出 50 個 等號
print("=" * 50)

# 一行一行讀出來
with open(file="./2-1.py", mode="r", encoding="utf-8") as f:
    for line in f:
        print(line) # 每一行後面會用 \n 結尾，所以輸出會有多次斷行的效果
```

註：

使用 open() 的時候，如果參數位置沒有改變，可以不用特別寫「file=… ，

mode=…」

寫入檔案

8-2.py

```
# 文字內容
text = '''人生短短幾個秋啊
不醉不罷休
東邊我的美人哪
'''

# 寫入檔案
with open("./8-2.txt", "w", encoding="utf-8") as f:
    f.write(text)

# 寫入最後一行 (附加文字在檔案內容最後一行)
with open("./8-2.txt", "a", encoding="utf-8") as f:
    f.write("西邊黃河流" + "\n")
```

刪除檔案

8-3.py

```
# 刪除檔案
import os # 匯入套件

# 檔案路徑
file_path = './8-2.txt'

# 如果指定路徑的檔案存在，則進行刪除
if os.path.exists(file_path):
    os.remove(file_path)
else:
    print('Not found')
```

參考資料

[1] Python File(文件) 方法

https://www.runoob.com/python/file-methods.html
[2] Python File Open
https://www.w3schools.com/python/python_file_handling.asp

Module 9. 模組

定義模組

將常用的程式碼（通常是函式）整合在一個 .py 檔案中，變成一個函式庫（library），或是重複使用的工具，在其它程式需要用到的時候，可以不用重新撰寫，只要透過 import 語法將其引入、匯入，即可在當前的程式檔案裡使用。

myModule.py

```
# 計算幾次方
def pow(x, y):
    return x**y

# 簡單斷句
def segmentSentence(text):
    list_sentences = text.split(' ')
    return list_sentences
```

使用模組

匯入模組的程式檔，通常是主程式；myModule.py 就是自訂模組，被主程式匯入，在主程式輸出 __name__ 內建變數的值，結果會是 __main__。

9-1.py

```
# 匯入模組
import myModule

if __name__ == '__main__':
    # 計算幾次方
    num = myModule.pow(2, 3)
    print(num)

    # 簡單斷詞
    txt = "I will always love you"
    list_result = myModule.segment(txt)
    print(list_result)
```

9-2.py

```python
# 直接從模組中匯入函式
from myModule import pow, segment

if __name__ == '__main__':
    # 計算幾次方
    num = pow(2, 3)
    print(num)

    # 簡單斷詞
    txt = "I will always love you"
    list_result = segment(txt)
    print(list_result)
```

補充：__name__ 內建變數的用法

__name__ 是內建變數，用來確認匯入模組的名稱（例如 myModule，會自動去掉 .py)，如果執行的主程式檔案裡面輸出 __name__ 變數的內容，它的值就會是 __main__。

myModule.py

```python
# 計算幾次方
def pow(x, y):
    return x**y

# 簡單斷句
def segmentSentence(text):
    list_sentences = text.split(' ')
    return list_sentences

# 想先在模組裡測試，再給其它程式檔案匯入
print( pow(7, 2) )
```

9-2.py

```python
# 直接從模組中匯入函式
from myModule import pow, segment

if __name__ == '__main__':
    # 計算幾次方
    num = pow(2, 3)
    print(num)

    # 簡單斷詞
    txt = "I will always love you"
    list_result = segment(txt)
    print(list_result)
```

如果想要測試自訂模組的執行結果是否如預期，可以執行 myModule.py：

```
(base) C:\Users\Owner\python_basics>python myModule.py
49

(base) C:\Users\Owner\python_basics>
```

圖： myModule.py 執行結果

此時執行 9-2.py 的話，會發生什麼事？

```
(base) C:\Users\Owner\python_basics>python 9-2.py
49
8
['I', 'will', 'always', 'love', 'you']

(base) C:\Users\Owner\python_basics>
```

圖： 突然輸出 49 這個模組內部測試結果

那是因為匯入模組的時候，Python 會將模組內部的程式碼 (myModule.py) 由上而下執行過一遍，連同模組的執行輸出也顯示在主要程式 (9-2.py) 裡。若是希望模組測試的結果，不要在主程式執行階段輸出，此時要將 myModule.py 改成：

myModule.py

```python
# 計算幾次方
def pow(x, y):
    return x**y

# 簡單斷句
def segmentSentence(text):
    list_sentences = text.split(' ')
    return list_sentences

'''
將模組當作主程式來執行，此時 myModule.py 的
__name__ 就會等於 __main__，
__main__ 就是主程式的意思
'''
if __name__ == "__main__":
```

```
    print( pow(7, 2) )
```

重新回到主程式 (9-2.py) 執行時，模組的輸出便不會顯示出來。

實務上，通常都會在撰寫程式碼的檔案加上「if __name__ == "__main__":」，來確保每個程式進行模組化 (modularization) 時，都能避免掉不必要的麻煩。

補充：模組不在當前路徑下的處理方式

倘若模組檔案不在當前的目錄下，可以整合 sys.path.insert() 方法來將指定目錄臨時設定在當前 python 版本可用的環境變數：

9-3.py

```
import sys, os
from pprint import pprint

# 取得當前 python 版本所使用的環境變數
pprint(sys.path)

# 輸出當前工作/專案/教材目錄的絕對路徑
print(os.getcwd())

# 設定絕對/相對路徑，取得自訂模組
sys.path.insert(0, os.getcwd() + '\\examples')
from myModule import pow, segment

'''
補充範例

如果在 python_basics 的上一層目錄裡，新增 modules 資料夾，
新增一個 tools.txt 記事本檔案，將 myModule.py 裡面的內容複製到 tools.txt 裡面，
而且把 tools.txt 的副檔名改成 py。

最後將前面的程式碼改成以下的樣子：
```

```
sys.path.insert(0, '..\\modules')
from tools import pow, segment
'''

# 確認環境變數是否有新增
pprint(sys.path)

if __name__ == '__main__':
    # 計算幾次方
    num = pow(2, 3)
    print(num)

    # 簡單斷詞
    txt = "I will always love you"
    list_result = segment(txt)
    print(list_result)
```

參考資料

[1] Python - if __name__ == '__main__' 涵義
http://blog.castman.net/%E6%95%99%E5%AD%B8/2018/01/27/python-name-main.html
[2] __main__ --- 最高层级代码环境
https://docs.python.org/zh-tw/3/library/__main__.html
[3] Python 中的 if __name__ == __main__
https://www.freecodecamp.org/chinese/news/if-name-main-python-example/

Module 10. 指令列參數

sys.argv 與 argparse.ArgumentParser 都是取得指令列 (command line) 參數 (arguments) 的方法，通常在執行程式碼的過程中，取得使用者輸入的資料。

sys.argv

10-1.py

```
# 匯入 sys 模組
import sys

# 確認 argv 的內容
print(sys.argv)
'''
請在 Terminal 使用以下範例指令:
python 10-1.py 100 3

輸出:
['10-1.py', '100', '3']
'''

# 簡單範例: 輸出 1 ~ 100 之間，3 的倍數
if len(sys.argv) < 3:
    print("參數量不足，至少需要 3 個參數。")
    print("使用方式: python 10-1.py <被除數> <除數>")

    '''
    sys.exit():
    中斷程式執行，直接結束程式，
    之後的程式碼不會繼續執行
    '''
    sys.exit()

for i in range(1, int(sys.argv[1])):
    if i % int(sys.argv[2]) == 0:
        print(i, end=" ")
```

argparse.ArgumentParser

10-2.py

```
# 匯入模組
from argparse import ArgumentParser

# 建立 AugumentParser 物件
parser = ArgumentParser()

# 加入參數與說明 (-D 和 --dividend 都是參數名稱)
parser.add_argument("-D", "--dividend", help="設定被除數",
default=100, type=int)
parser.add_argument("-d", "--divisor", help="設定除數", default=3,
type=int)
parser.add_argument("--msg", help="輸出說明文字", default="取得 3 的倍數
", type=str)

# 取得使用者資料 (來自指令參數)
args = parser.parse_args()
'''
註 1:
要撰寫 parser.parse_args()，
執行指令「python 10-2.py --help」才有效

註 2:
執行指令「python 10-2.py -D 100 -d 3 --msg 輸出 1 到 100 之間 3 的倍數」時，
-D 的資料，要用 args.dividend 去取值，
-d 的資料，要用 args.divisor 去取值，
--msg 的資料，因為沒有設定縮寫名稱，需要直接使用 args.msg
'''
# 需要先提示使用方式的，可以設定 parser.print_help()
parser.print_help()

# 輸出使用者資料
print(args.dividend, args.divisor, args.msg)
```

```python
# 簡單範例
print("=" * 50)
print(f"輸出訊息: {args.msg}")
for i in range(1, args.dividend):
    if i % args.divisor == 0:
        print(i, end=" ")
```

參考資料

[1] Python 命令列參數 sys.argv 使用教學與範例
https://officeguide.cc/python-sys-argv-command-line-arguments-tutorial-examples/

[2] Python 超好用標準函式庫 argparse
https://dboyliao.medium.com/python-超好用標準函式庫-argparse-4eab2e9dcc69

[3] argparse — Parser for command-line options, arguments and sub-commands
https://docs.python.org/3/library/argparse.html

Module 11. 修飾子 Decorator

有時候需要將函式 (function) 當作參數，以物件形態傳遞到另一個函式當中來運作，此時接收函式物件的**函式**，稱之為修飾子 (decorator)，或稱裝飾器、修飾符、修飾器。

基本概念

11-1.py

```
# 建立一個函式 price_a，回傳 100
def price_a():
    return 100

# 第一個是 function 物件，第二個是執行 function (回傳 100) 的結果
print(price_a)
print(price_a())

# 建立函式 price_b，作為 decorator，
# 收到函式物件，並在 price_b 執行該函式物件
def price_b(func):
    '''
    將 func (price_a) 執行後所回傳的值 (100)，
    乘上 1.2 後，再回傳
    '''
    return func() * 1.2

# 修飾子執行結果
result = price_b(price_a)
print(result)
```

在大多的程式語言中，某種語法對程式功能沒有影響，卻更方便開發者使用，同時讓程式更加簡潔，且有更高的可讀性，這種語法稱之為語法糖 (syntactic sugar)，python 在修飾子的使用上，以「@」+ 修飾子(函式)名稱，作為語法糖。

使用語法糖的修飾子

11-2.py

```
# 建立函式 price_b (decorator)
def price_b(func):
    '''
    將 func (price_a) 執行後所回傳的值 (100)，
    乘上 1.2 後，再以暱名函式回傳
    '''
    return lambda: func() * 1.2

# python 以「@」作為修飾子符號
@price_b
def price_a():
    return 100

# 修飾子執行結果
print(price_a())
```

修飾子可以疊加

11-3.py

```
'''
使用 @ 語法的修飾子可以「由下往上」疊加，
將 @price_c 放到 @price_b 上方，
再執行、輸出 price_a() 的結果看看。
'''

# 建立函式 price_b (decorator)
def price_b(func):
    '''
    將 func (price_a) 執行後所回傳的值 (100)，
    乘上 1.2 後，再以暱名函式回傳
    '''
    return lambda: func() * 1.2

# 用來疊加的修飾子
```

```python
def price_c(func):
    '''
    將 func (price_b) 執行後所回傳的值 (120)，
    乘上 2 後，再以暱名函式回傳
    '''
    return lambda: func() * 2

    # 除了暱稱函式外，常見還有以下寫法
    '''
    def tempFunc():
        return func() * 2
    return tempFunc
    '''

# python 以「@」作為修飾子符號
@price_c
@price_b
def price_a():
    return 100

# 修飾子執行結果
print(price_a())
```

參考資料

[1] [Python 教學] decorator 裝飾詞原理到應用
https://www.maxlist.xyz/2019/12/07/python-decorator/
[2] Python 進階技巧 (3) — 神奇又美好的 Decorator ，嗷嗚！
https://medium.com/citycoddee/python 進階技巧-3-神奇又美好的-decorator-嗷嗚-6559edc87bc0
[3] 裝飾器
https://openhome.cc/zh-tw/python/meta-programming/decorator/
[4] [工作上的筆記] python 裝飾器、屬性裝飾器、類裝飾器
https://ithelp.ithome.com.tw/articles/10312168

Module 12. 安裝套件

pip 指令

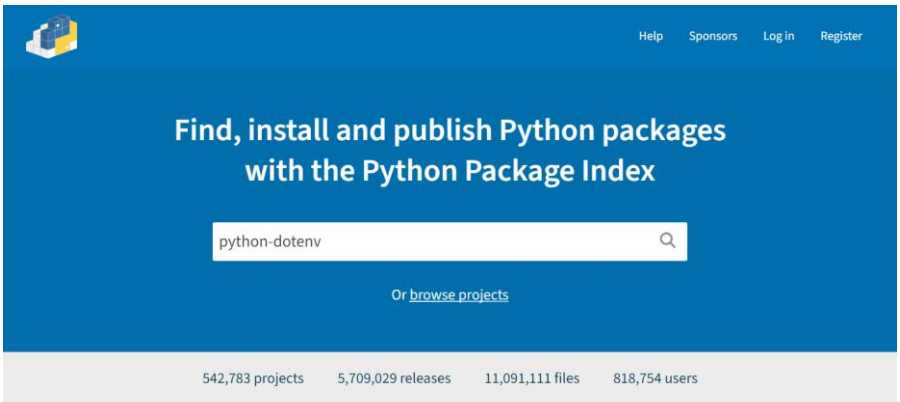
他人在網路上公開的模組或工具，通常稱之為套件 (Package，有些人會稱為第三方元件 Third Party)，需要透過 pip 指令來進行安裝。

pip 全名是 The Python Package Installer，是一種安裝管理套件的工具，原則上要使用 python 相關的套件，會使用 pip 來安裝或是更新 python 相關套件。在 conda 環境裡，預設已經安裝好 pip 程式，

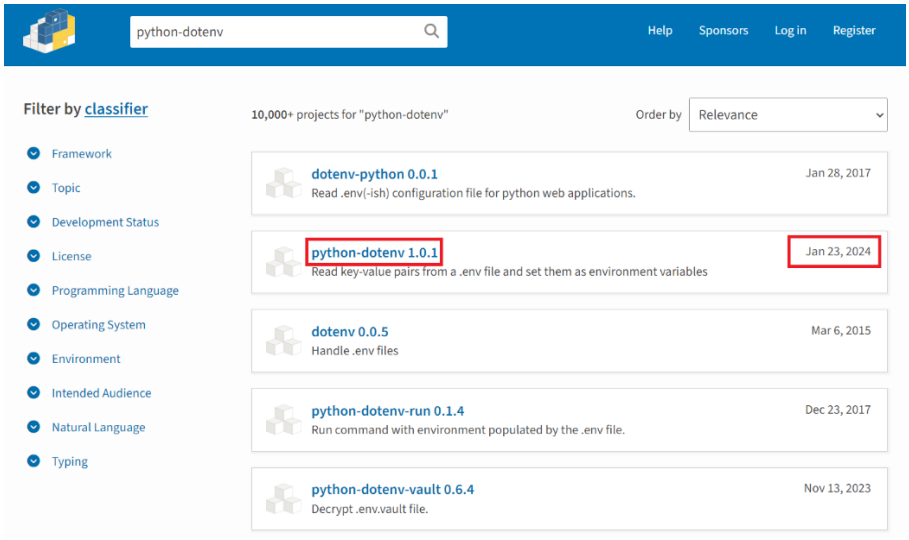
語法

`# 安裝套件`
`pip install <package-name>`
 	
`# 更新套件`
`pip install -U <package-name>`
 	
`# 安裝套件列表 (通常會將套件清單放在 requirements.txt)`
`pip install -r requirements.txt`

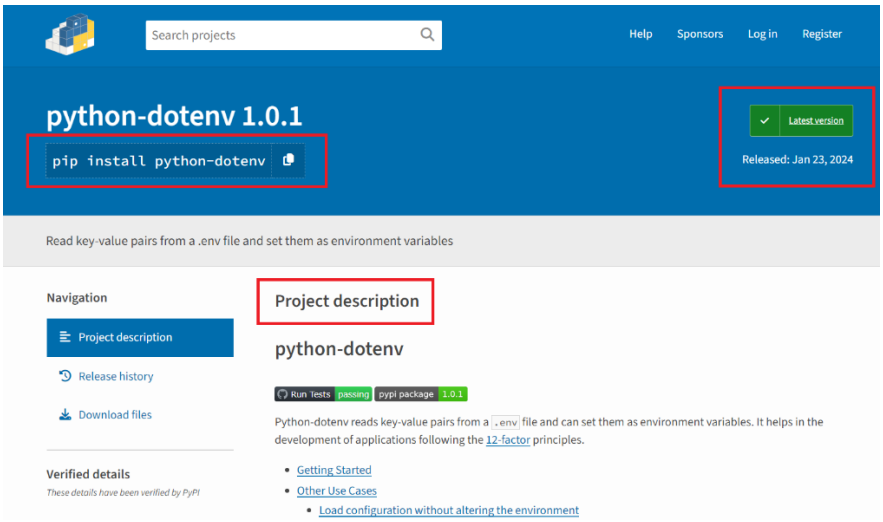
可以到 https://pypi.org/ 搜尋套件，例如搜尋「python-dotenv」。



圖：在搜尋欄位輸入「python-dotenv」，再按下 Enter



圖：找一個時間離現在近一點的，這裡的時候是指套件更新、釋出時間



圖：左邊是安裝套件的指令，右邊是最後釋出時間，下方是套件相關說明

請在當前 conda 環境下的 Terminal，輸入以下指令，並按下 Enter：
`pip install python-dotenv`

新增 .env 檔
`USERNAME=darren` `PASSWORD=123456`

註：「.檔案或目錄名稱」通常是隱藏的，作為系統和程式所需的配置設定檔。

12-1.py
`from dotenv import load_dotenv, dotenv_values`

```python
import os

# 讀取 .env 檔案 (使用 load_dotenv)
load_dotenv()

# 取得 .env 檔案內容 (內容的變數名稱可以是小寫，也可以是大寫)
username = os.getenv('USERNAME')
password = os.getenv('PASSWORD')
print(username, password)

# 讀取 .env 檔案 (使用 dotenv_values)
config = dotenv_values(".env")

# 取得 .env 檔案內容 (內容的變數名稱可以是小寫，也可以是大寫)
print(config) # config = {'USERNAME': 'darren', 'PASSWORD': 123456}
print(config['USERNAME'], config['PASSWORD'])
```

參考資料

[1] 【Python 教學】pip install 指令大全
https://www.maxlist.xyz/2019/07/13/pip-install-python/
[2] Find, install and publish Python packages with the Python Package Index
https://pypi.org/
[3] python-dotenv
https://pypi.org/project/python-dotenv/